

**FACULTY
OF MATHEMATICS
AND PHYSICS**
Charles University

BACHELOR THESIS

Šimon Kurz

A lightweight containerization tool for evaluation of programming assignments

Department of Software Engineering

Supervisor of the bachelor thesis: RNDr. David Bednárek, Ph.D.

Study programme: Computer Science

Prague 2025

I declare that I carried out this bachelor thesis on my own, and only with the cited sources, literature and other professional sources. I understand that my work relates to the rights and obligations under the Act No. 121/2000 Sb., the Copyright Act, as amended, in particular the fact that the Charles University has the right to conclude a license agreement on the use of this work as a school work pursuant to Section 60 subsection 1 of the Copyright Act.

In date
Author's signature

Title: A lightweight containerization tool for evaluation of programming assignments

Author: Šimon Kurz

Department: Department of Software Engineering

Supervisor: RNDr. David Bednárek, Ph.D., Department of Software Engineering

Abstract: Automated evaluation of programming assignments includes execution of code that is not trusted; therefore, we need a protective mechanism that protects the evaluating system from malicious or inadvertent actions of the program being evaluated. In addition, some assignments may require restrictions on available libraries or services or precise measurement of resource utilization. Such requirements are typically solved with containers; however, the well-known mainstream options, like Podman and Docker, are not sufficiently flexible and lightweight for this purpose. The goal of this thesis is the implementation of a lightweight utility that provides the required isolation directly, using the Namespace and Cgroup services of modern Linux kernels. Since these services are not completely well documented, the thesis shall involve investigation of the available documentation and experiments; thus, the thesis shall also provide insights into the containerization aspects of Linux. The utility shall be designed to work with ReCoDex, as a replacement and extension of its current process-isolation subsystem.

Keywords: automated evaluation, Linux, containers, C++

Název práce: Odlehčený kontejnerizační nástroj pro vyhodnocování programovacích úloh

Autor: Šimon Kurz

Katedra: Katedra softwarového inženýrství

Vedoucí bakalářské práce: RNDr. David Bednárek, Ph.D., Katedra softwarového inženýrství

Abstrakt: Automatizované vyhodnocování programovacích úloh zahrnuje spouštění neznámého kódu, a proto vyžaduje ochranný mechanismus, který zajistí bezpečnost vyhodnocovacího systému vůči škodlivému nebo chybovému chování testovaného programu. Některé úlohy navíc vyžadují omezení dostupných knihoven či služeb, nebo přesné měření využití systémových prostředků. Tyto požadavky se obvykle řeší pomocí kontejnerů, avšak běžně používané nástroje jako Docker či Podman nejsou pro tento účel dostatečně flexibilní a jejich režie představuje vysokou zátěž pro systém. Cílem této práce je návrh a implementace odlehčeného nástroje, který zajistí potřebnou izolaci přímo, pomocí mechanismů Namespaces a Cgroups moderního Linuxového jádra. Vzhledem k neúplné dokumentaci těchto funkcí práce zahrnuje studium dostupných zdrojů a experimentální ověřování chování systému. Výsledkem je nejen samotný nástroj, ale i hlubší pohled na aspekty kontejnerizace v Linuxu. Nástroj je navržen pro integraci do systému ReCodEx jako náhrada a rozšíření jeho stávajícího mechanismu izolace procesů.

Klíčová slova: automatizované vyhodnocování, Linux, kontejnery C++

Contents

Introduction	7
1 Automated evaluation of programming assignments - ReCoDex	10
1.1 ReCoDex workflow	10
1.2 Motivation for using containerization	10
1.3 Motivation for a new containerization tool	11
2 Linux and its features for containerization	13
2.1 Filesystem	13
2.2 Permissions and Access Rights	14
2.3 Process model	14
2.3.1 User and Group IDs	14
2.3.2 Process Control and Communication	15
2.3.3 Environment Variables	16
2.4 Management of resources	16
2.4.1 Resource Limits (rlimit)	16
2.4.2 Control Groups (cgroups)	16
2.5 Process isolation using Namespaces	17
2.5.1 Mount Namespaces	18
2.5.2 Cgroup Namespaces	18
2.5.3 PID namespaces	19
2.5.4 Other namespace types	19
3 ReCoDex Isolator - new containerization tool for ReCoDex	21
3.1 Terminology	21
3.2 General flow of operation	21
3.2.1 Initializing global resources	21
3.2.2 The proxy process	22
3.2.3 Configuring new namespaces	23
3.2.4 Creating the sandbox filesystem tree	23
3.2.5 Launching and overseeing tasks	24
3.3 The complete sandbox environment	25
3.4 Implementation, code structure	25
3.4.1 Initial setup (root_core)	26
3.4.2 Assigning instance identifiers (root_credentials_manager)	27
3.4.3 Initializing the cgroup hierarchy (root_cgroup_manager)	28
3.4.4 Implementation details of cgroup filesystem interaction (cgroupv2_t)	29
3.4.5 Launching the proxy process (proxy_connector)	31
3.4.6 Setting up the sandbox from the proxy process (proxy_core)	34
3.4.7 Setting up the mount namespace (proxy_mount_manager)	36

3.4.8	Creating the sandbox filesystem view (box_fs_manager)	39
3.4.9	Applying directory rules (dir_rule_supervisor)	40
3.4.10	Enabling controllers for tasks (proxy_cgroup_manager)	41
3.4.11	Managing sandbox credentials (proxy_credentials_manager)	42
3.4.12	Orchestrating tasks (task_manager)	43
3.4.13	Supervising task execution (task_supervisor)	44
4	Testing the ReCoDex Isolator	50
4.1	Testing Basic Isolation and Resource Limits	50
4.2	Testing on ReCoDex assignments	51
4.2.1	ReCodEx Worker	51
4.2.2	Mocking the ReCodEx Worker	51
4.3	Pitfalls of containerized environments	51
4.3.1	Compiling C/C++ using gcc/g++	52
4.3.2	Offline compilation of C# .NET	53
5	Future work	55
	Conclusion	57
	Bibliography	58
A	Attachments	60
A.1	ReCoDex Isolator project repository	60
A.1.1	Running automated testing scripts	60
A.1.2	Running the tool manually	62

Introduction

Automated evaluation of programming assignments includes execution of code that is not trusted; therefore, a protective mechanism that protects the evaluating system from malicious or inadvertent actions of the program being evaluated. In addition, some assignments may require restrictions on available libraries or services or precise measurement of resource utilization. Such requirements are typically solved with containers; however, the well-known mainstream options, like Podman and Docker, are not sufficiently flexible and lightweight for this purpose.

The goal of this thesis is the implementation of a lightweight utility that provides the required isolation directly, using namespace and cgroup services of modern Linux kernels. Since these services are not completely well documented, the thesis shall involve investigation of the available documentation and experiments; thus, the thesis shall also provide insights into the containerization aspects of Linux. The utility shall be designed to work with ReCoDex, as a replacement and extension of its current process-isolation subsystem called Isolate.

ReCoDex

ReCoDex is a system for automated evaluation of programming assignments. Students and teachers interact with ReCoDex through a web interface. Teachers can create and manage assignments, and students submit their solutions. The system then automatically compiles, runs the students' code in a set of predefined test cases, and sends the results back to the students.

Running untrusted code

Running student solutions means running unknown, potentially harmful programs on a server. This is why they are run in an environment isolated from the rest of the system. On Linux, this isolation is achieved with tools that create so called **containers**.

In our context, a **container** could be defined as *a lightweight isolated environment that packages an application together with its dependencies and allows us to run it easily on different machines*¹. A container is not a full virtual machine - it does not have its own operating system. But in today's Linux, a very high degree of isolation is possible - it is possible for an application in a container to see its own view of the filesystem, network, environment variables, user IDs, process IDs, etc. These are useful both for portability and security - the application is launched in a controlled environment, and (hopefully) cannot interact with parts of the system we do not want it to.

¹<https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/>

Achieving containerization in Linux

To achieve proper isolation and security while still providing the necessary resources for programs to run, there is a broad spectrum of responsibilities a container runtime needs to fulfill. There are two main subsystems of the Linux kernel that make it feasible in the first place:

- **Namespaces** - The main Linux mechanism for isolation and virtualization of system resources. There are eight types of namespaces, each allowing us to virtualize different components of the system.
- **Cgroups** - The main mechanism for managing resource usage. Cgroups allow hierarchical grouping of processes, assigning limits on resource usage to each group, and viewing accounting data, like the total time spent on the CPU or the amount of used memory by processes in the group.

Cgroups and namespaces are essential, but they represent only a subset of the functionality required to build a working container. A complete solution must combine *cgroups*, *namespaces*, and additional kernel features and system calls in a carefully coordinated way. The available kernel mechanisms often overlap in functionality, and their documentation is often incomplete. As a result, the development process requires not only the study of existing documentation, but also significant experimentation. Specific kernel features that were found relevant are described throughout the following chapters.

Note about references and bibliography

Most of the information regarding Linux behavior in this thesis comes directly from manual pages, kernel documentation, and practical experimentation. To keep the bibliography concise while still crediting useful documentation, a distinction has been made:

- *Key documentation of larger kernel subsystems, important syscalls, relevant software projects and academic works* are included in the **Bibliography**.
- *Less critical references, such as manual pages of some syscalls*, are cited inline using **footnotes**.

ReCoDex Isolator - the new utility

Widely used containerization tools already exist, e.g., Podman or Docker. However, especially in the case of resource measurement, these tools do not offer the flexibility required by ReCoDex. Currently, a smaller containerization tool is used - Isolate by Martin Mares and Bernard Blackham [1].

We are developing the new utility (see Attachment A.1) as a more modern replacement of Isolate, engineered specifically for ReCoDex. It draws inspiration from Isolate, and could be described as a modernization and extension of it. It is written in C++ 23, using Cgroups version 2, CMake, and contemporary C++ libraries. Special emphasis is put on proper modularity and documentation, so that it becomes easy to use, easily extensible and modifiable for future changes.

The tool is now ready for further work on integration into the ReCoDex system. By simulating a simplified version of the ReCoDex backend, it was successfully tested on a set of real assignment configurations. It is well documented and modularized, and should serve as a good base for future extensions.

1 Automated evaluation of programming assignments - ReCoDex

In its documentation, ReCoDex is described as follows [2]:

ReCodEx is a system for testing people’s programming skills. Our primary target are technical universities. These often have programming courses that require the students to write programs that have to be checked by teachers. By checking the basic properties (the source code compiles and the program works correctly for a defined set of inputs) automatically, ReCodEx lets teachers focus on more important parts of the course, while also providing students with continuous feedback.

The system is highly customizable - it allows testing simple programs in C, but also in languages such as Java, Python, Prolog or Haskell. Even complicated scenarios such as testing parsers based on flex, yacc and bison or running highly parallel programs on a NUMA machine are feasible.

1.1 ReCoDex workflow

Both students and teachers interact with ReCodEx through a web interface. Teachers can create and manage assignments, and students submit their solutions. After receiving a solution from a student, ReCoDex performs these steps:

- deliver the solution to the server
- compile the solution
- run the solution and capture its output for each test case
- evaluate the solution’s correctness by comparing expected and real outputs
- report results of the evaluation back to the student

The backend component of ReCoDex responsible for compiling, running and evaluating solutions is the Worker [3]. For each test case, it orchestrates the process of compiling, launching the solution and running so called Judges to evaluate correctness.

1.2 Motivation for using containerization

In ReCoDex, compilation of the solution, the solution itself and the evaluation are run in controlled, isolated environments (currently using Isolate [1]). There are several reasons why to run the evaluated program inside of a containerized environment.

Security

Any harmful code, even if unintentionally, can be submitted, and its execution should not affect other students, the grading system, or the host infrastructure. Secure design of automatic grading systems is discussed by one of the authors of Isolate, Martin Mareš [4], and running the graded program inside a containerized environment is considered essential, besides other design considerations. Containerization allows student programs to run in a controlled environment and restricting possible attack channels. This prevents both accidental and intentional harm, including denial-of-service attacks or unauthorized data access.

Reproducibility of Results

The grading process should be reproducible and deterministic to ensure fairness. If compiling and running submissions is not decoupled from the configuration of the host, differences in library versions, system updates, or configuration changes could cause solutions to behave differently over time. By controlling the execution environment, including available binaries, libraries, and filesystem contents, we make sure that all submissions are evaluated under similar conditions.

Resource Management

The amount of resources consumed by the program, such as CPU time, memory usage, or disk space, is often a part of the grading criteria. The Linux kernel provides mechanisms for limiting and measuring resource usage via cgroups and other mechanisms. Although it is possible to enforce resource limits without full containerization, integrating resource control into a sandboxed environment gives us additional options and control.

Modularity and Portability

Containerization improves modularity by allowing different assignments to specify their own runtime environments, including language versions, compilers, and libraries. This decouples the grading environment from the host system configuration, allowing assignments that rely on specific versions of compilers or languages to remain valid across years of operation.

1.3 Motivation for a new containerization tool

Although ReCoDex has successfully relied on the Isolate tool, it has some limitations. Isolate was not originally designed with ReCoDex or programming assignments in mind. It is specifically being developed for the *International Olympiad in Informatics (IOI)*¹.

The new containerization tool presented in this thesis is designed specifically for integration with ReCoDex. It directly leverages the capabilities of modern Linux kernels. In particular, it is built around Cgroups version 2, which offers a

¹<https://ioinformatics.org/>

more unified and modern interface for resource management, compared to earlier versions.

An important design goal was modularity. The tool is structured so that the core responsibilities of the tool, such as management of cgroups, namespaces, or the filesystem, are clearly separated and well documented. Extensive documentation comes with the implementation, making it more accessible for future contributors. This simplifies maintenance, debugging and extending the tool for future requirements and new types of assignments. For example, enabling controlled networking between sandboxes should be possible without major redesign.

2 Linux and its features for containerization

In order to allow a better understanding of the concepts presented in following chapters about the tool's design and operation, this chapter quickly introduces Linux terminology and features relevant to process isolation and containers.

2.1 Filesystem

One of the main ideas in Unix-like operating systems is that *everything is a file*. This includes not just files in the usual sense (text files, binaries) but also directories, devices, sockets, kernel interfaces, etc. The same system calls like `open()`, `read()`, and `write()` are used to interact with most of the system.

The filesystem is hierarchical, starting at the root directory `/`. Usual top-level directories include:

- `/bin`, `/usr/bin` – user binary programs
- `/etc` – configuration files
- `/dev` – devices
- `/proc`, `/sys` – kernel interfaces exposed as files
- `/tmp`, `/var/tmp` – temporary files

Some of these, like `/proc` and `/sys`, are *virtual filesystems*. They do not exist physically on a disk, but are generated by the kernel at runtime. For example, the virtual file `/proc/self/status` shows information about the process that reads it.

Devices are also represented as files, typically located in `/dev`. This allows user-space programs to interact with hardware using the same file-based interface. There are also 'virtual' devices under `/dev` - for example, writing to `/dev/null` discards data, and reading from `/dev/urandom` produces random bytes.

Mounting and Filesystem Views

Linux allows *mounting*¹ new filesystems or existing directories anywhere in the filesystem tree. It is possible to build a completely custom filesystem view by masking a directory with a virtual filesystem or the contents of another physical directory. Containers rely on this - by setting up a new root (e.g., with `pivot_root()`²), we can give the process a filesystem where only a chosen subset of the host's filesystem is visible.

¹<https://man7.org/linux/man-pages/man2/mount.2.html>

²https://man7.org/linux/man-pages/man2/pivot_root.2.html

2.2 Permissions and Access Rights

Every file has an owner (identified by UID), an associated group (identified by GID), and a set of permissions determining who (which processes) can read/edit or execute the file.

The permission model is based on three sets of bits:

- **User** (u) – permissions for processes with the UID of the file’s owner
- **Group** (g) – permissions for processes of the file’s group
- **Other** (o) – permissions for all other processes

For example, `-rwxr-x--` means full access for the owner, read/execute access for the group, and no access for others. These permissions are enforced by the kernel whenever a process tries to access a file.

Directories also use the same permission bits. Execute permission on a directory means that files inside the directory can be accessed if we know the filename, read permission allows listing its contents, and write permission allows creating or deleting files inside.

2.3 Process model

A process is the fundamental unit of program execution. It represents a running instance of a program - encapsulating the code, its current state (program counter, registers), memory, and system resources associated to it, such as file descriptors and environment variables.

Processes are identified by a unique **Process ID** (PID). They are organized hierarchically, and a process can spawn one or more *child processes* using the `fork()` system call. These parent-child relationships form a process tree. The first process on the system (typically `init` or `systemd`³) has PID 1 and serves as the root of this tree.

A process can replace its code and memory space with a different program using the `execve()` system call (or one of its wrappers). A combination of `fork()`⁴ followed by `execve()`⁵ is the classical way of creating processes in Linux.

2.3.1 User and Group IDs

Every process in Linux runs with a real and effective user ID and group ID. To put it simply, the real IDs determine **who you are** and the effective IDs **what you can do**, especially in relation to file permissions. Simpler container runtimes rely on switching to a non-root UID inside the container to restrict the process’s privileges. A more sophisticated approach involves creating a new

³<https://wiki.archlinux.org/title/Systemd>

⁴<https://man7.org/linux/man-pages/man2/fork.2.html>

⁵<https://man7.org/linux/man-pages/man2/execve.2.html>

`user namespace`⁶ to remap UIDs between the host and container. This allows a process to run as UID 0 inside the container (which some software expects), while still being unprivileged from the host's perspective.

Beyond file permissions, the kernel has features like `capabilities`⁷ and `seccomp`⁸ to restrict what a process can do.

2.3.2 Process Control and Communication

Process Termination and Waiting

When a process finishes, it exits with an exit code, either explicitly via `exit()`⁹ or implicitly when it returns from `main()`. The parent process can observe this using `waitpid()`¹⁰, which returns the PID of the terminated child and fills a status variable with information about how the child exited.

Signals¹¹

Signals are an important IPC mechanism, providing asynchronous notifications to processes. A signal is a simple integer value delivered by the kernel to a target process or thread, which may interrupt its normal execution. Processes can register custom *signal handlers* to react to specific signals or choose to ignore them. Certain signals cannot be overridden (e.g., `SIGKILL` for immediate termination).

Examples of other IPC mechanisms:

- `Unix domain sockets`¹²: Bidirectional, reliable communication channels between processes on the same host.
- `Message queues`¹³: Queues for passing discrete messages with priorities.
- `Semaphores`¹⁴: Synchronization primitives for coordinating access to shared resources.
- `Shared memory`¹⁵: Memory regions mapped into multiple processes, allowing data sharing with minimal overhead.
- `Pipes and FIFOs`¹⁶: Byte streams allowing unidirectional or bidirectional communication between processes.

⁶https://man7.org/linux/man-pages/man7/user_namespaces.7.html

⁷<https://man7.org/linux/man-pages/man7/capabilities.7.html>

⁸<https://man7.org/linux/man-pages/man2/seccomp.2.html>

⁹<https://linux.die.net/man/2/exit>

¹⁰<https://linux.die.net/man/2/waitpid>

¹¹<https://man7.org/linux/man-pages/man7/signal.7.html>

¹²<https://man7.org/linux/man-pages/man7/unix.7.html>

¹³https://man7.org/linux/man-pages/man7/mq_overview.7.html

¹⁴https://man7.org/linux/man-pages/man7/sem_overview.7.html

¹⁵https://man7.org/linux/man-pages/man7/shm_overview.7.html

¹⁶<https://man7.org/linux/man-pages/man7/pipe.7.html>

2.3.3 Environment Variables

Environment variables¹⁷ are key-value pairs used to influence program behavior without requiring extra command-line arguments or configuration files. Some common examples:

- `PATH` – used by shells and `execvp()`¹⁸ to locate executables
- `PWD` - current working directory
- `HOME` – home directory of the current user

When a process is created via `fork()` and `execve()`, it receives the set of environment variables as an array of strings from its parent. Each string is in the form `KEY=value`. The kernel does not interpret environment variables — using them is entirely up to userspace programs.

Environment variables are per-process and do not require any special namespace isolation - each process has its own environment array.

2.4 Management of resources

Linux provides multiple mechanisms to restrict and monitor the resource usage of processes.

2.4.1 Resource Limits (`rlimit`)

At the basic level, Linux allows resource usage to be measured and limited by using the `getrusage()`, `setrlimit()` and `getrlimit()` system calls [5]. These limits can restrict values such as the maximum size of a process's address space, the number of open file descriptors, or the amount of CPU time the process is allowed to consume.

`rlimit` operates on a per-process basis and limits are inherited across `fork()` - unless changed, the child process gets the same resource limits as the parent.

`rlimit` provides no mechanism for managing and measuring cumulative resource usage of process hierarchies.

2.4.2 Control Groups (`cgroups`)

To provide control over resource allocation and monitoring across groups of processes, the Linux kernel includes a subsystem called **control groups**, or **cgroups** [6]. Cgroups allow processes to be organized into hierarchical groups for which resource limits, scheduling priorities, and accounting can be managed.

Different types of resources are managed with different **controllers**, such as:

- `cpu`: controls and accounts CPU usage.

¹⁷https://wiki.archlinux.org/title/Environment_variables

¹⁸<https://www.man7.org/linux/man-pages/man3/exec.3p.html>

- **memory**: limits and accounts memory usage.
- **pids**: limits the number of processes that can be created.
- **io**: manages I/O bandwidth.

The cgroup subsystem is controlled via a virtual filesystem mounted at `/sys/fs/cgroup`. Individual cgroups are created by creating new directories in this filesystem. Processes are added to cgroups by writing their PIDs to the appropriate `cgroup.procs` file, and resource limits are set by writing to controller-specific files.

For example, limiting memory usage of a group to 1 MB can be done by calling:

```
echo 1000000 > /sys/fs/cgroup/<group>/memory.max
```

In addition to enforcing limits, cgroups provide detailed accounting statistics. Files like `memory.current` or `cpu.stat` expose data about the resource usage of a cgroup.

2.5 Process isolation using Namespaces

The Linux kernel includes a mechanism called *namespaces* best described by the description in their manual page [7]:

A namespace wraps a global system resource in an abstraction that makes it appear to the processes within the namespace that they have their own isolated instance of the global resource. Changes to the global resource are visible to other processes that are members of the namespace, but are invisible to other processes. One use of namespaces is to implement containers.

There are 8 types of namespaces, each isolating different types of resources:

- **pid**: Virtualizes PIDs. Processes in a PID namespace see only other processes in the same namespace. The first process in a namespace gets PID 1, and acts similarly as if it was the `init` process of that namespace.
- **mount**: Isolates the filesystem mount points, allowing processes to see different filesystem hierarchies.
- **uts**: Isolates hostname and domain name, useful for giving different identities to containers.
- **net**: Isolates resources associated with networking, including network interfaces, routing tables, and firewall rules.
- **ipc**: Isolates System V IPC objects and POSIX message queues.
- **user**: Virtualizes UID/GID mappings, allowing processes to have root privileges within a namespace while remaining unprivileged outside of it.

- **cgroup**: Provides isolated views of the cgroup hierarchy.
- **time**: Virtualizes the system clocks, allowing processes to see different values for `CLOCK_MONOTONIC` and `CLOCK_BOOTTIME`.

A new namespace is created by calls to `clone()`¹⁹, or `unshare()`²⁰, and usually is deleted when the last process in the namespace either terminates or leaves the namespace. Processes can be moved between namespaces using `setns()`²¹.

When designing the containerization tool, some namespaces showed to be particularly important:

2.5.1 Mount Namespaces

Mount namespaces [8] isolate the set of mount points visible to a process. When a process is placed in a new mount namespace, it starts with a copy of the parent namespace's mount table. By default, subsequent changes to mount points, such as mounting or unmounting filesystems, only affect the new namespace and not the rest of the system.

This functionality is very useful for creating environments with a customized view of the filesystem. Mount namespaces allow us to:

- Hide parts of the filesystem from the process (e.g., by not mounting them).
- Mount temporary filesystems (e.g., `tmpfs`²²) to prevent writes to disk.
- Use `bind mounts` to relocate files and directories to different paths or mounting directories read-only.
- Change the root of the filesystem using `pivot_root()`²³, creating an illusion of a root directory somewhere else in the filesystem.

2.5.2 Cgroup Namespaces

Cgroup namespaces virtualize the view of a process's cgroups as seen through `/proc/pid/cgroup` and `/proc/pid/mountinfo`, and the overall view of the cgroup hierarchy as seen through `/sys/fs/cgroup`. The manual page for cgroup namespaces [9] describes these benefits of virtualizing cgroup hierarchies:

- It prevents information leaks whereby cgroup directory paths outside of a container would otherwise be visible to processes in the container. Such leakages could, for example, reveal information about the container framework to containerized applications.
- It eases tasks such as container migration. The virtualization provided by cgroup namespaces allows containers to be isolated from knowledge of the pathnames of ancestor cgroups.

¹⁹<https://man7.org/linux/man-pages/man2/clone.2.html>

²⁰<https://man7.org/linux/man-pages/man2/unshare.2.html>

²¹<https://man7.org/linux/man-pages/man2/setns.2.html>

²²<https://docs.kernel.org/filesystems/tmpfs.html>

²³https://man7.org/linux/man-pages/man2/pivot_root.2.html

- It allows better confinement of containerized processes, because it is possible to mount the container’s cgroup filesystems such that the container processes cannot gain access to ancestor cgroup directories.

The details of how cgroup namespaces work are complicated and described using examples in the manual page. For our purposes, an important finding was that by using the `clone3()`²⁴ syscall with the `CLONE_INTO_CGROUP` and `CLONE_NEWCGROUP` flags, we can change the root of the new cgroup namespace to the specified cgroup.

2.5.3 PID namespaces

PID namespaces [10] isolate the process ID number space. When a process is placed into a new PID namespace, it becomes PID 1 within that namespace and starts a new process hierarchy. Processes can only see and interact with other processes in the same namespace (or child namespaces).

Just like the system-wide PID 1, the process with PID 1 in a namespace has special significance. There are special rules for delivering signals to it, which we need to be aware of when communicating with a containerized process. When PID 1 exits, the entire namespace is invalidated and all processes within it are killed.

2.5.4 Other namespace types

The rest of the namespaces are currently not configured explicitly (beyond creating a new namespace inside the sandbox to add isolation), but they can be relevant for future extensions.

Note on Network namespaces

Network namespaces [11] provide each namespace with its own isolated network stack. This includes network interfaces, routing tables, firewall rules, port spaces, etc. Processes within one network namespace cannot interact with the network of other namespaces unless explicitly connected by virtual interfaces.

In the current implementation of the new tool, network namespaces are used only to ensure isolation. The default configuration of a new network namespace only contains the loopback interface, which prevents any interaction with the host network or between concurrently running sandboxes.

However, network namespaces also allow more complex configurations. A possible future extension to our tool is controlled networking between sandboxed processes. This would require setting up `veth pairs`²⁵ and bridging or routing traffic between namespaces.

²⁴<https://man.archlinux.org/man/clone3.2.en>

²⁵<https://man7.org/linux/man-pages/man4/veth.4.html>

Note on User namespaces

User namespaces [12] are a special type of Linux namespace that enable a process to have different user and group IDs inside the namespace than on the host system. Each non-user namespace is "owned" by a user namespace. Most importantly, they allow processes to run with UID 0 (root) inside the namespace while remaining unprivileged from the host's perspective.

At the moment, the new tool does not make use of user namespaces.

3 ReCoDex Isolator - new containerization tool for ReCoDex

3.1 Terminology

Before going further, it is necessary to clarify some terms and concepts specific to the Isolator tool and this thesis. These terms are used throughout the chapter to describe the architecture, implementation, and operation of the Isolator.

- **Instance:** A single run of the Isolator tool created for sequential execution of one or more related tasks. Each instance has its own dedicated resources and identifiers to prevent interference between concurrent instances.
- **Sandbox:** A restricted execution environment created by the tool, designed to limit the actions and resource usage of untrusted code.
- **Root Process:** The initial process responsible for setting up global resources and launching the isolated environment. It coordinates creation of **instance**-specific namespaces, cgroups, and filesystem structures.
- **Proxy Process:** A process running inside the initial isolated environment (namespaces and cgroups) that performs further setup, such as configuring the filesystem view, and launches user tasks.
- **Task Process:** The actual process that executes the untrusted code (e.g., a student submission) within the sandboxed environment, subject to all configured restrictions and resource limits.

3.2 General flow of operation

In this section, we will overview the general flow of operation of the tool. The focus will be on the procedures needed to create the sandboxed environment and running processes inside it, we will skip general setup like parsing configuration.

The operation is carried out by three different processes - **root process**, **proxy process**, and **task processes**. Very briefly, the root process is responsible for parsing configuration and reserving non-isolated global resources, the proxy process performs the setup of the sandboxed environment, and the task processes run the isolated programs in this environment.

3.2.1 Initializing global resources

The ultimate goal is to run processes in an isolated environment that are able to access or influence only a dedicated part of the system. First, we need to allocate the isolated resources that the instance will operate on. For some of the system resources, **namespaces** do this by default, by creating multiple copies of the same resource. However, some resources have to exist globally in the system.

In our case: the sandboxed filesystem tree, a cgroup hierarchy to apply limits and accounting, and a distinct UID/GID to limit the sandboxed processes' privileges (because we do not use `user namespaces`[12]). For each Isolator instance, these have to be allocated manually, and we need to prevent colliding with an existing filesystem/cgroup path or UID/GID.

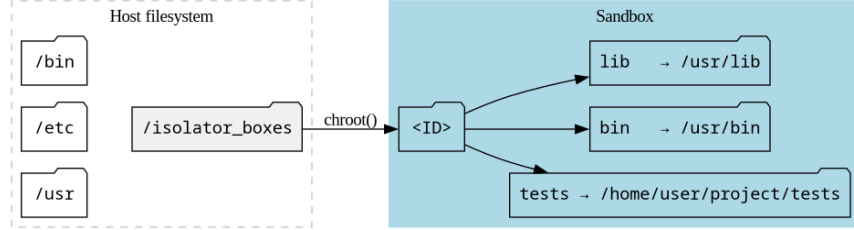


Figure 3.1 The filesystem structure of an instance

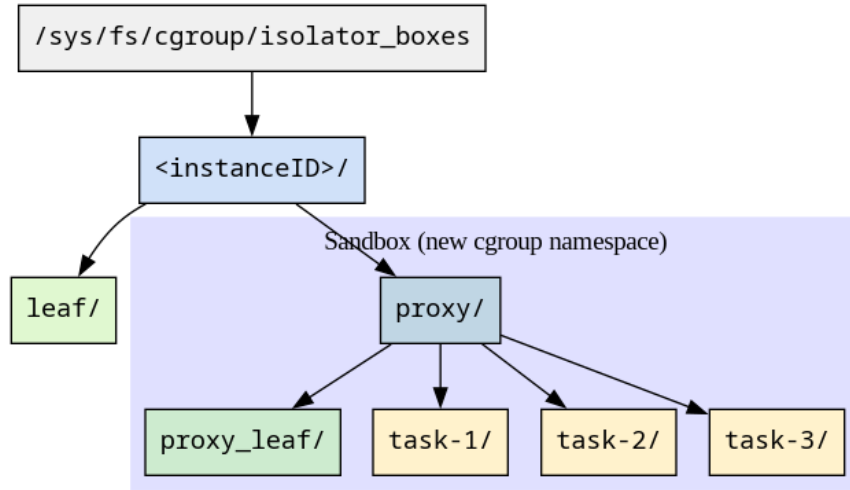


Figure 3.2 The cgroup hierarchy of an instance

To prevent colliding with the rest of the system, we dedicate a subtree of the host filesystem and of the host cgroup hierarchy to sandboxes created by our tool. Individual instances then exist under these paths.

To differentiate between **instances** and prevent collisions between them, a unique integer is allocated to each instance - an **instance ID**. The filesystem and cgroup paths, and UIDs/GIDs for an instance are then derived from this ID.

3.2.2 The proxy process

After global resources are allocated and initialized, the root process delegates operation to the **proxy process**. The proxy process is launched in new namespaces - the same namespaces in which the sandboxed process will run. So, in some aspects, it is already isolated and performs the setup of the environment from inside the sandbox. However, the isolation is still very limited, as it sees all of the filesystem (And it has to, to be able to create the custom sandboxed filesystem view).

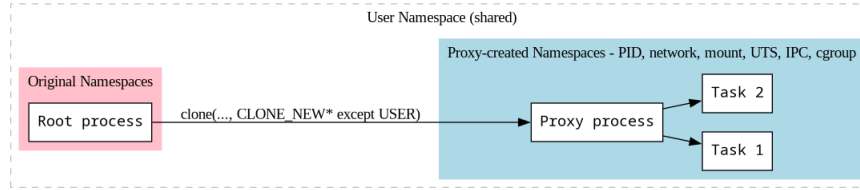


Figure 3.3 The namespace setup of an instance

3.2.3 Configuring new namespaces

Each namespace type has different semantics, some are very configurable and the degree of necessary configuration depends on the exact behaviour we expect.

One namespace that, in our case, requires nontrivial additional configuration, is the `mount namespace`[8]. For example, we allow propagation of mount events from the host into the sandbox, but not the other way around. A large source of possible security vulnerabilities comes from the fact that we let isolated processes access directories on the host through `mount()`[13]. The exact security properties depend on how the mount namespace is set up and which mounts are performed.

Some namespaces create isolated copies of system resources (like the `UTS namespace` with the hostname and domain name). If a process in the new namespace tries changing the hostname, nothing will happen to the hostname visible from the original namespace. In a case like this, we do not need a lot of additional configuration (possibly just changing these names in the new namespace if we do not want them revealed).

3.2.4 Creating the sandbox filesystem tree

After proper configuration of namespaces, we are ready to create the isolated directory tree visible from the sandbox. This is currently the most complex responsibility of the proxy process. It relies on making directories on the host system visible inside the sandbox (using `mount()` calls), based on the configuration provided by the user.

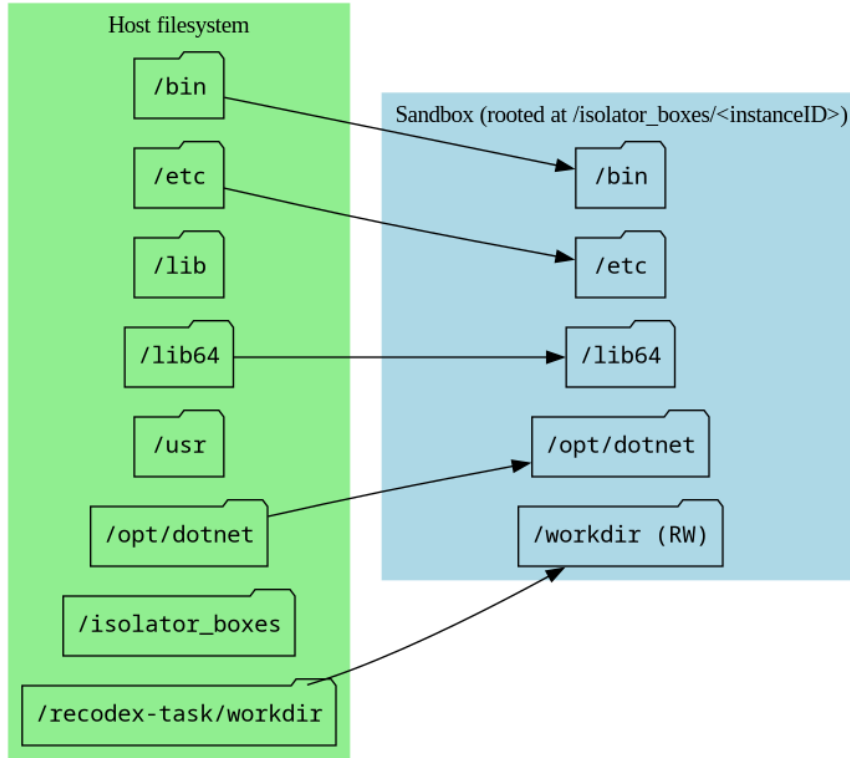


Figure 3.4 Typical sandbox filesystem setup

The user specifies which directories or virtual filesystems will be visible inside the sandbox, under which paths. The usual setup in ReCoDex is a set of default read-only directories required for most Linux programs to run (`/lib`, `/lib64`, `/etc`, ...), ReCoDex specific directories (like `/opt/dotnet` with a custom dotnet installation), and a single read-write directory (where the sandboxed UID/GID has all permissions) provided by ReCoDex, which is then inspected for output files to provide evaluation.

After the sandbox filesystem is set up, we make its root (`/isolate_boxes/<instanceID>`) the root for the proxy process. At this point, after running the process in separate namespaces and restricting it to a specific filesystem subtree, the sandboxed environment is almost complete. Task processes inherit this environment, and the rest of the setup is specific for each task process.

3.2.5 Launching and overseeing tasks

Tasks are specified by

- A path to the executable program (e.g., binary, or a shell script)
- An array of command line arguments for the executable
- A set of environment variables
- Configuration of resource limits
- Configuration of output redirection

The task processes are created from the proxy process and placed into their dedicated cgroup by using `clone()`. Before doing this, the proxy process creates the cgroup, and configures its resource limits. After cloning, the final, process-specific setup is performed inside the task process:

- `stdout` and `stderr` are optionally redirected to user specified files
- resource limits not handled by cgroups are set using `setrlimit()`
- UID and GID are switched to the values assigned to the sandbox, to prevent access where not allowed

After this setup, control of the task process is handed over to the supplied executable by calling `execve()`¹. `execve()` also passes the array of command line arguments and the set of environment variables, which was the last missing component of the isolated environment. While a task process is running, the proxy process waits for its termination. If a task process violates some of the restrictions of the sandbox, it is killed, and the proxy receives details about the termination.

3.3 The complete sandbox environment

The environment setup performed by the tool means the sandboxed processes run in an environment made of these components:

- Private namespaces created while cloning the proxy, isolating or virtualizing various system resources.
- Private subtree of the filesystem, where only specified directories from the host are visible or even writable. Common for all tasks in a single instance, same as for the namespaces.
- Resource limits enforced by cgroups, like the maximum amount of used memory or number of created child processes.
- Other resource limits, like the maximum amount of CPU time, enforced by `setrlimit[5]`.
- An assigned UID/GID that limits the process's access rights and privileges.
- An explicitly specified set of environment variables.

3.4 Implementation, code structure

In this section, we will explain how the operation flow described above is reflected in the code and decomposition into classes. This section is much more detailed and introduces additional steps that were skipped earlier.

The code samples presented are written in Python style pseudocode to omit C++ clutter and showcase important concepts more clearly. They were generated from the source files using ChatGPT²

¹<https://man7.org/linux/man-pages/man2/execve.2.html>

²<https://chatgpt.com/>

3.4.1 Initial setup (root_core)

The `root_core` class implements the responsibilities of the *root process*. These include parsing the configuration, preparing global resources for a sandbox instance (instance ID, UID/GID, instance cgroup, instance root directory), and starting the proxy process.

Listing 3.1 Pseudocode of `root_core`

```
class root_core:
    def __init__(self, argc, argv):
        self.root_config = root_configuration(argc, argv)
        self.credentials = root_credentials_manager(self.root_config.
            get_credentials_config())
        self.cg_mgr = root_cgroup_manager(self.root_config, self.credentials)
        self.proxy_connector = proxy_connector(self.root_config, self.
            credentials, self.cg_mgr)

    def run(self):
        self.credentials.run()
        self.create_sandbox_root_dir()
        self.cg_mgr.run()
        self.proxy_connector.run_and_wait_for_proxy()
```

Class Members:

- `root_configuration`: Contains the parsed configuration data for the instance.
- `root_credentials_manager`: Reserves sandbox-specific identifiers (Instance ID, UID/GID) and provides UID/GID switching.
- `root_cgroup_manager`: Initializes the cgroup hierarchy for the sandbox instance.
- `proxy_connector`: Launches the proxy process into its own set of namespaces.

Overview of operation:

The `run()` method represents the `main()` function of the whole instance. It performs the following steps:

1. Reserve sandbox credentials by calling `credentials.run()`.
2. Create the root filesystem directory for the sandbox via `create_sandbox_root_dir()`. This directory will serve as the root directory for the isolated filesystem tree of the sandbox.
3. Initialize the sandbox cgroup hierarchy by calling `cg_mgr.run()`.
4. Launch and oversee the proxy process through `proxy_connector.run_and_wait_for_proxy()`.

The root process exits after waiting for the proxy process. This allows it to remain as a supervisor for the sandbox instance and perform cleanups at the end.

3.4.2 Assigning instance identifiers (root_credentials_manager)

The `root_credentials_manager` class is responsible for assigning instance-unique identifiers. This includes reserving a unique `instance_id`, computing the sandbox UID and GID used for filesystem ownership, and determining the filesystem root and cgroup paths dedicated to this instance. The actual switching between credentials (UID/GID) is handled later inside the proxy process.

Listing 3.2 Pseudocode of `root_credentials_manager`

```
class root_credentials_manager:
    def __init__(self, config):
        self.config = config

    def run(self):
        self.orig_uid = getuid()
        self.orig_gid = getgid()
        self.assign_ids()
        self.box_root = self.assign_box_root(self.box_id)
        self.box_cgroup = self.assign_box_cgroup(self.box_id)

    def assign_ids(self):
        if self.config.instance_id:
            self.instance_id = self.config.instance_id
        else:
            self.instance_id = random_integer(1, 5000)

        if self.config.box_uid:
            self.box_uid = self.config.box_uid
        else:
            self.box_uid = self.box_id + 60000

        if self.config.box_gid:
            self.box_gid = self.config.box_gid
        else:
            self.box_gid = self.box_uid

    def assign_box_root(self, box_id):
        idf = self.config.instance_name or str(box_id)
        return self.config.boxes_dir / idf

    def assign_box_cgroup(self, box_id):
        idf = self.config.instance_name or str(box_id)
        return self.config.boxes_cgroup / idf
```

Class Members:

- `config`: Configuration node for the credentials manager.
- `instance_id`: Unique identifier assigned to this sandbox instance.
- `box_uid`, `box_gid`: UID and GID assigned for sandbox ownership.
- `orig_uid`, `orig_gid`: UID and GID of the user who launched the instance.

- `box_root`: Filesystem path assigned for this sandbox's root directory.
- `box_cgroup`: Path for this sandbox's root cgroup.

Overview of operation:

When `run()` is called:

1. The original UID and GID of the launching process are recorded.
2. The `assign_ids()` method selects a sandbox ID. If not explicitly configured, a random ID in the range 1-5000 is generated.
3. The sandbox UID and GID are derived from the `instance_id`. By default, UID assignment uses an offset of 60000.
4. The filesystem root directory and cgroup path are constructed based on the assigned `instance_id` or instance name.

The current implementation selects a random `instance_id` if none is provided. In the future, this could be replaced by a dedicated credential allocation service to avoid collisions automatically.

3.4.3 Initializing the cgroup hierarchy (`root_cgroup_manager`)

The `root_cgroup_manager` class is responsible for initializing the cgroup hierarchy for each sandbox instance. The detailed logic of cgroup manipulation is encapsulated in a helper class `cgroupv2_t` and described in subsection 3.4.4.

Listing 3.3 Pseudocode of `root_cgroup_manager`

```
class root_cgroup_manager:
    def __init__(self, config, credentials):
        self.config = config
        self.credentials = credentials

    def run(self):
        instance_cg = self.credentials.instance_cgroup()
        self.instance_cgrp = cgroupv2_t(instance_cg)
        self.leaf_cgrp = cgroupv2_t(instance_cg / "leaf")
        self.proxy_cgrp = cgroupv2_t(instance_cg / "proxy")

        self.leaf_cgrp.add_me()
        self.root_cgrp.enable_all_cntrlrs()
        self.instance_cgrp.enable_all_cntrlrs()

    def open_proxy_fd(self):
        return self.proxy_cgrp.open_fd()

    def close_proxy_fd(self):
        self.proxy_cgrp.close_fd()
```

Class Members:

- `config`: Configuration node for the root cgroup manager.
- `credentials`: Reference to the credentials manager that stores the instance-specific cgroup path.
- `root_cgrp`: Interface for the root cgroup mount (`/sys/fs/cgroup`).
- `instance_cgrp`: Cgroup assigned to this sandbox instance.
- `proxy_cgrp`: Cgroup of the sandbox cgroup namespace.
- `leaf_cgrp`: Cgroup where the root process is placed.

Overview of operation:

When `run()` is invoked:

1. The filesystem paths for instance, leaf, and proxy cgroups are derived from the credentials.
2. The cgroups are created by instantiating `cgroupv2_t` objects.
3. The root process is moved into the leaf cgroup.
4. Controllers are enabled in the correct order: first globally, then for the instance subtree.

The file descriptor returned by `open_proxy_fd()` is later (in the `proxy_connector`, subsection 3.4.5) passed to the `clone3()` syscall with the `CLONE_INTO_CGROUP` flag to directly assign the proxy process to its cgroup during creation.

3.4.4 Implementation details of cgroup filesystem interaction (`cgroupv2_t`)

The `cgroupv2_t` class encapsulates operations related to a single cgroup:

- Creation of the cgroup during initialization.
- Opening file descriptors for use with `CLONE_INTO_CGROUP`.
- Enabling the `cpu`, `memory`, and `pids` controllers.
- Adding processes into the cgroup (by writing their PID into `cgroup.procs`).
- Reading usage statistics and applying resource limits.

The controllers are wrapped by additional classes (`cpu_cntrlr`, `memory_cntrlr`, `pid_cntrlr`).

Listing 3.4 Pseudocode of `cgroupv2_t`

```
class cgroupv2_t:
    def __init__(self, cgrp_path):
        self.cgrp_path = CGROUP_FS_PATH / rel_cgrp_path
        self.cpu = cpu_cntrlr(self.cgrp_path)
        self.mem = memory_cntrlr(self.cgrp_path)
        self.pid = pid_cntrlr(self.cgrp_path)
        self.fd = None
        self.init()

    def init(self):
        if directory_exists(self.cgrp_path):
            terminate("Cgroup already exists")
        if not create_directory(self.cgrp_path):
            terminate("Creating cgroup failed")

    def open_fd(self):
        self.fd = open(self.cgrp_path, O_DIRECTORY | O_RDONLY)
        return self.fd

    def enable_all_controllers(self):
        if not (self.cpu.enable() and self.mem.enable() and self.pid.enable()):
            terminate("Failed to enable controllers")

    def add_me(self):
        mypid = getpid()
        write_to_file(self.cgrp_path / "cgroup.procs", mypid)

    def set_strict_memory_limit(self, bytes):
        self.mem.set_memory_max(bytes)
        self.mem.set_memory_min_to_max()

    def set_processes_limit(self, n):
        self.pid.set_pids_max(n)
```

The `cgroupv2_t` class operates by interacting with the cgroup v2 virtual filesystem. On instantiation, it creates a new directory representing the cgroup, using `std::filesystem::create_directory()` for its full path (`/sys/fs/cgroup/<group>`).

To enable controllers for child cgroups, the controller classes write the corresponding controller type (e.g., `+cpu`) into `cgroup.subtree_control`. Adding processes to a cgroup is handled by writing their PID into `cgroup.procs`.

3.4.4.1 Example: Setting strict memory limits

Configuring memory usage limits using cgroupv2 showed to be less straightforward than expected.

In the documentation of cgroupv2, the cgroup `memory.max` file is described as follows [6]:

Memory usage hard limit. This is the main mechanism to limit memory usage of a cgroup. If a cgroup's memory usage reaches this limit and can't be reduced, the OOM killer is invoked in the cgroup. Under certain circumstances, the usage may go over the limit temporarily.

And an excerpt from the description of `memory.min`:

Hard memory protection. If the memory usage of a cgroup is within its effective min boundary, the cgroup's memory will not be reclaimed under any conditions.

During our experiments with `swap memory`³ enabled, the kernel appeared to begin reclaiming memory, allowing the process to continue running even after reaching `memory.max`. This behavior is consistent with the description of `memory.max`. However, setting `memory.min` to "max", which should disable memory reclaim, did not help - the behavior was exactly the same as before.

We achieved consistent expected results only after turning swap off. In a setup that combined `memory.max`, `memory.min` and `sudo swapoff -a`, once `memory.current` approached the `memory.max` threshold, the kernel terminated the process.

The following example shows the working configuration. An equivalent approach is used in the C++ code of our tool.

Listing 3.5 Creating and configuring a cgroup with strict memory limits

```
# Disable swap to ensure enforcement of memory.max
swapoff -a

# Assume cgroup v2 is mounted at /sys/fs/cgroup

# Create a new sandbox hierarchy
mkdir /sys/fs/cgroup/my_sandbox
mkdir /sys/fs/cgroup/my_sandbox/task1

# Enable memory controller in the parent
echo "+memory" > /sys/fs/cgroup/my_sandbox/cgroup.subtree_control

# Disable memory.min protection
echo max > /sys/fs/cgroup/my_sandbox/task1/memory.min

# Set strict hard memory limit (e.g., 256 MiB)
echo 268435456 > /sys/fs/cgroup/my_sandbox/task1/memory.max

# Add a process (the current shell) to the cgroup
echo $$ > /sys/fs/cgroup/my_sandbox/task1/cgroup.procs
```

3.4.5 Launching the proxy process (`proxy_connector`)

The `proxy_connector` class is responsible for launching and waiting for the proxy process. The proxy is started using the `clone3()` system call, and is directly placed into the appropriate cgroup.

³<https://wiki.archlinux.org/title/Swap>

Listing 3.6 Pseudocode of proxy_connector

```
class proxy_connector:
    def __init__(self, root_config, credentials, cg_manager):
        self.root_config = root_config
        self.credentials = credentials
        self.cg_manager = cg_manager

    def run_and_wait_for_proxy(self):
        pid = self.spawn_proxy()
        self.wait_for_proxy(pid)

    def spawn_proxy(self):
        proxy_conf = self.root_config.get_proxy_config()
        pid = clone3_proxy(proxy_conf, self.cg_manager.open_proxy_fd())

        if pid < 0:
            terminate("Clone3 failed")

        elif pid == 0:
            self.cg_manager.close_proxy_fd()
            proxy = proxy_core(proxy_conf, self.credentials)
            proxy.run()
            terminate("Unexpected return from proxy")

        self.cg_manager.close_proxy_fd()
        return pid

    def wait_for_proxy(self, pid):
        status = waitpid(pid)
        if status.failed:
            terminate("waitpid failed")
        log("Proxy exited", status)

    def proxy_clone_args(self, proxy_conf, cgrp_fd):
        args = clone_args()
        args.exit_signal = SIGCHLD
        args.flags = (
            CLONE_NEWIPC | CLONE_NEWNS | CLONE_NEWPID |
            CLONE_NEWCGROUP | CLONE_NEWUTS | CLONE_INT0_CGROUP
        )
        if not proxy_conf.share_net:
            args.flags |= CLONE_NEWNET
        args.cgroup = cgrp_fd
        return args

    def clone3_proxy(self, config, cgrp_fd):
        args = self.proxy_clone_args(config, cgrp_fd)
        return syscall_clone3(args)
```

Class members:

- **root_config:** Root configuration node containing configuration for the proxy.
- **credentials:** Provides credentials for UID/GID switching inside the proxy.

- `cg_manager`: Provides the cgroup file descriptor for `CLONE_INTO_CGROUP`.

Execution flow:

When `run_and_wait_for_proxy()` is called:

1. `spawn_proxy()` invokes `clone3()` with appropriate namespace and cgroup flags.
2. The proxy process enters its cgroup, and starts executing the `proxy_core`, setting up the sandbox environment and running tasks.
3. The root process waits for the proxy to terminate using `waitpid()`.

Use of `clone3()`:

The `proxy_connector` uses `clone3()` [14] to create the proxy process. Compared to the traditional `clone()` or `fork()` calls, `clone3()` allows more control over the created process.

Listing 3.7 Code sample showcasing `clone3` usage

```
clone_args proxy_clone_args(const config::proxy_config& proxy_conf, uint64_t
    cgrp_fd)
{
    clone_args args{0};
    args.exit_signal = SIGCHLD;

    args.flags = CLONE_NEWIPC | CLONE_NEWNS |
                 CLONE_NEWPID | CLONE_NEWCGROUP | CLONE_NEWUTS |
                 CLONE_INTO_CGROUP;
    if(!proxy_conf.share_net()) args.flags |= CLONE_NEWNET;
    args.cgroup = cgrp_fd;
    return args;
}

pid_t clone3_proxy(const config::proxy_config& config, uint64_t cgrp_fd)
{
    auto args = proxy_clone_args(config, cgrp_fd);
    return syscall(SYS_clone3, &args, sizeof(clone_args));
}
```

In our case, `clone3()` is called with the following flags:

- `CLONE_NEWIPC`: Creates a new IPC namespace.
- `CLONE_NEWNS`: Creates a new mount namespace.
- `CLONE_NEWPID`: Creates a new PID namespace.
- `CLONE_NEWCGROUP`: Creates a new cgroup namespace.
- `CLONE_NEWUTS`: Creates a new UTS namespace.
- `CLONE_NEWNET` (optional): Creates a new network namespace if network sharing is disabled in the configuration.

- `CLONE_INT0_CGROUP`: Assigns the child process directly into the cgroup whose file descriptor is passed via `args.cgroup`. In combination with `CLONE_NEWCGROUP`, this makes this cgroup the root of the new namespace.

The `clone3()` call returns:

- A positive PID in the parent (root process).
- Zero in the child (proxy process).
- Negative value in case of an error (e.g., invalid flags, insufficient privileges).

Inside the proxy process (when `pid == 0`), the `proxy_core` is instantiated and ran. The root process waits for the proxy using `waitpid()`.

Use of `waitpid()`:

After spawning the proxy, the parent process calls `waitpid()`⁴ to synchronously wait for the proxy process to terminate. The exit status returned by `waitpid()` includes both the exit code and the signal that caused termination (if any). If `waitpid()` itself fails, a fatal error is reported.

3.4.6 Setting up the sandbox from the proxy process (`proxy_core`)

The `proxy_core` class is responsible for setting up the sandboxed environment from inside the proxy process, after it was launched by `proxy_connector`.

Listing 3.8 Pseudocode of `proxy_core`

```
class proxy_core:
    def __init__(self, proxy_config, root_credentials_manager):
        self.proxy_config = proxy_config
        self.credentials = proxy_credentials_manager()
        self.mount_mgr = proxy_mount_manager()
        self.fs_manager = box_fs_manager()
        self.env_manager = env_manager()
        self.cg_mgr = proxy_cgroup_manager()
        self.task_manager = task_manager()

    def run(self):
        self.proxy_env_setup()
        self.task_manager.run_all_tasks()
        exit(0)

    def proxy_env_setup(self):
        self.mount_mgr.run()
        self.fs_manager.run()
        self.pivot_root()
        self.cg_mgr.run()

    def pivot_root(self):
        box_root = self.credentials.box_root()
        syscall_pivot_root(box_root, box_root)
        chdir("/")
        umount2(box_root, MNT_DETACH)
```

⁴<https://linux.die.net/man/2/waitpid>

Class members:

- `proxy_config`: Proxy configuration node.
- `mount_mgr`: Responsible for setting up the mount namespace.
- `cg_mgr`: Prepares the proxy-local cgroup hierarchy.
- `fs_manager`: Prepares the filesystem tree inside the sandbox.
- `env_manager`: Constructs environment variables for task processes.
- `credentials_manager`: Manages switching credentials for task processes.
- `task_manager`: Manages task launching and evaluation.

Execution flow:

1. The constructor initializes its member classes that perform environment setup and help execute tasks.
2. `proxy_env_setup()` runs the mount namespace setup, filesystem preparation, changes the root directory using `pivot_root()`, and initializes cgroups.
3. Tasks are launched and executed using the `task_runner`.

Use of `pivot_root()`:

The `pivot_root()` [15] system call is used to change the visible filesystem root of the running process (and its child processes) to the prepared sandboxed directory tree.

Listing 3.9 Usage of `pivot_root`

```
void pivot_root()
{
    auto& box_root = credentials_mgr.box_root();
    chdir(box_root.c_str());
    if(syscall(SYS_pivot_root, ".", "."))
        { terminate("pivot_root() failed, errno: {}", errno); }

    /// MNT_DETACH ensures we don't get a busy error.
    if(umount2(".", MNT_DETACH))
        { terminate("umount() on old root failed, errno: {}", errno); }
}
```

The setup used here is explained on the manual page as follows:

`new_root` and `put_old` may be the same directory. In particular, the following sequence allows a pivot-root operation without needing to create and remove a temporary directory:

```
chdir(new_root);
pivot_root(".", ".");
umount2(".", MNT_DETACH);
```

This sequence succeeds because the `pivot_root()` call stacks the old root mount point on top of the new root mount point at `/`. At that point, the calling process's root directory and current working directory refer to the new root mount point (`new_root`). During the subsequent `umount()` call, resolution of `"."` starts with `new_root` and then moves up the list of mounts stacked at `/`, with the result that old root mount point is unmounted.

3.4.7 Setting up the mount namespace (`proxy_mount_manager`)

The `proxy_mount_manager` class is responsible for preparing the mount namespace for the sandboxed environment. It ensures that mounts performed inside the sandbox do not propagate to the host system, sets up the mount point necessary for `pivot_root()`, and remounts the cgroup filesystem into the sandbox.

Listing 3.10 Pseudocode of `proxy_mount_manager`

```
class proxy_mount_manager:
    def __init__(self, proxy_config, credentials):
        self.proxy_config = proxy_config
        self.credentials = credentials

    def run(self):
        self.make_root_rslave()
        self.mount_pivot_dir()
        self.mount_cgroup()

    def make_root_rslave(self):
        if mount(None, "/", None, MS_SLAVE | MS_REC, None):
            terminate("failed to change propagation type of root mount")

    def mount_pivot_dir(self):
        box_root = self.credentials.box_root()
        if mount(box_root, box_root, None, MS_REC | MS_BIND, None):
            terminate("failed to bind mount the sandbox root directory")

    def mount_cgroup(self):
        box_cg_root = self.credentials.box_root() / "sys/fs/cgroup"
        create_directories(box_cg_root)

        if umount("/sys/fs/cgroup"):
            terminate("failed to unmount cgroup filesystem")

        if mount("none", box_cg_root, "cgroup2", 0, None):
            terminate("failed to remount cgroup2 filesystem")
```

Class Members:

- `proxy_config`: Configuration node for the proxy environment.
- `credentials`: Reference to the credentials manager holding the box root path.

Overview of operation:

When `run()` is called:

1. The mount propagation of the root filesystem is changed to `RSLAVE`, preventing mounts inside the sandbox from propagating to the host or other namespaces.
2. The box root directory is bind-mounted onto itself, which is required for the `pivot_root()` operation.
3. The cgroup v2 filesystem is remounted inside the sandbox.

3.4.7.1 Usage of `mount()`

This class nicely showcases three different ways `mount()` [13] can be used.

Changing propagation of mount events

The function `make_root_rslave` modifies propagation of mount events, by calling the `mount()` system call with special flags:

```
void make_root_rslave()
{
    // don't propagate mount events to other namespaces
    if(mount(nullptr, "/", nullptr, MS_SLAVE | MS_REC, nullptr))
        { terminate(); }
}
```

When used like this, `mount()` does not mount a new filesystem or directory. Instead, it changes the propagation type of an existing mount point. The parameters are:

- **source:** `nullptr` - no directory is being mounted.
- **target:** `"/"` - the mount point to change.
- **filesystemtype:** `nullptr` - no filesystem is being mounted.
- **mountflags:** `MS_SLAVE / MS_REC` - recursively set the propagation type to slave.
- **data:** `nullptr` - unused here.

`MS_SLAVE` sets the mount propagation mode to *slave*: mount events from the parent namespace propagate into this namespace, but not outside of it.

`MS_REC` applies this recursively to all mounts under the root of the sandbox.

Bind mounting a directory

```
void mount_pivot_dir()
{
    // the directory that we pivot_root to has to be a mount point
    auto& box_root = credentials_>box_root();
```

```

    if(mount(box_root.c_str(), box_root.c_str(), nullptr, MS_REC |
        MS_BIND, nullptr))
    {
        terminate("failed to bind mount the pivot directory, errno:
            {} ", errno);
    }
}

```

This function prepares the directory that will later become the sandbox root. The kernel requires the target of `pivot_root()` to be a mount point, so the code performs a bind mount of the directory to itself.

- source: `box_root.c_str()` - the directory to be bind mounted.
- target: `box_root.c_str()` - same as source.
- filesystemtype: `nullptr` - no new filesystem is being mounted.
- mountflags: `MS_BIND | MS_REC` - perform a recursive bind mount.
- data: `nullptr` - unused here.

Mounting a virtual filesystem

```

void mount_cgroup()
{
    if(umount(cgroup::CGROUP_FS_PATH().c_str()))
    {
        terminate("failed to unmount cgroup filesystem, errno: {}",
            errno);
    }

    if(mount("none", (box_cg_root).c_str(), "cgroup2", 0, nullptr))
    {
        terminate("failed to remount cgroup2 filesystem, errno: {}",
            errno);
    }
}

```

Inside the new mount namespace, this function unmounts the global `cgroup2` mount at `/sys/fs/cgroup` in the host filesystem and mounts a new instance into the sandbox. Task handlers later use it to set resource limits and retrieve accounting.

Parameters passed to mount `cgroup2`:

- source: `"none"`, because `cgroup2` is a virtual filesystem.
- target: `box_cg_root.c_str()` - the path inside the sandbox.
- filesystemtype: `"cgroup2"` - the type of filesystem to mount.
- mountflags: `0` - no special flags needed.
- data: `nullptr` - unused here.

3.4.8 Creating the sandbox filesystem view (box_fs_manager)

The `box_fs_manager` class is responsible for constructing the sandbox's directory tree. The tree is built according to a set of **directory rules** that describe which host paths should be mounted into which locations inside the sandbox.

Listing 3.11 Pseudocode of `box_fs_manager`

```
class box_fs_manager:
    def __init__(self, fs_config, credentials):
        self.fs_config = fs_config
        self.credentials = credentials

    def run(self):
        self.construct_box_tree()

    def construct_box_tree(self):
        self.create_mount_points()
        self.apply_rules()

    def create_mount_points(self):
        for rule in self.fs_config.rules():
            self.create_directories_for_rule(rule)

    def apply_rules(self):
        for rule in self.fs_config.rules():
            dir_rule_supervisor(rule, self.credentials).apply()
```

Class Members:

- `fs_config`: Configuration node describing the directory rules.
- `credentials`: Reference to the credentials manager to set correct ownership of created directories.

Overview of operation:

When `run()` is called:

1. The sandbox root directory is prepared based on the configuration.
2. `create_mount_points()` ensures that all target directories exist. Dummy directories are created if needed to allow nested mount points.
3. `apply_rules()` iterates over all directory mapping rules and applies bind mounts and ownership changes as needed.

The `apply_rule` step is dedicated to a helper class, `dir_rule_supervisor`.

3.4.9 Applying directory rules (dir_rule_supervisor)

The `dir_rule_supervisor` class is responsible for applying an individual directory rule. Each rule describes how a directory or a filesystem should be mounted from the host into the sandbox. The supervisor creates any missing directories, sets up ownership and permissions, and performs the actual bind mounts or filesystem mounts based on the rule type.

Listing 3.12 Pseudocode of `dir_rule_supervisor`

```
class dir_rule_supervisor:
    def __init__(self, rule, credentials):
        self.rule = rule
        self.credentials = credentials

    def apply(self):
        if self.rule.maybe and not is_directory(self.rule.out_dir):
            return

        in_path = self.credentials.box_root / self.rule.in_dir
        out_path = self.rule.out_dir
        flags = self.default_flags()

        if self.rule.fs:
            mount("none", in_path, out_path[1:], flags, "")
            if in_path == "proc":
                mount("none", in_path, out_path[1:], flags | MS_REMOUNT, "
                    hidepid=2")
        else:
            flags |= MS_BIND | MS_NOSUID
            if not self.rule.norec:
                flags |= MS_REC
            mount(out_path, in_path, "none", flags, "")
            mount(out_path, in_path, "none", MS_REMOUNT | flags, "")
            if self.rule.rw:
                chmod(out_path, 0777)
                for entry in recursive_directory_iterator(out_path):
                    chmod(entry, 0777)

    def create_dir(self, dir):
        if is_directory(dir):
            terminate("Directory already exists: {}", dir)
        create_directories(dir)
        if self.rule.tmp:
            chown(dir, self.credentials.box_uid, self.credentials.box_gid)
            chmod(dir, 0700)
```

Class Members:

- `rule`: Configuration of a single directory rule describing how to map a host path or a filesystem into the sandbox.
- `credentials`: Reference to the credentials manager, for setting UID/GID ownership.

Overview of operation:

When a rule is applied:

1. If the rule has `maybe()` set and the outside directory does not exist, it is skipped.
2. If the rule mounts a filesystem (`fs()` flag), the mount is performed directly (e.g., mounting `procfs`).
3. Otherwise, the rule performs a bind mount of an external directory into the sandbox.
4. For writable mounts (with `rw()` set) permissions are updated to allow access for task processes.

3.4.10 Enabling controllers for tasks (`proxy_cgroup_manager`)

The `proxy_cgroup_manager` class is responsible for configuring the controllers inside the proxy's cgroup namespace. After the proxy process has been launched into its root cgroup using `CLONE_INTO_CGROUP` and `CLONE_NEWCGROUP`, this manager enables controllers for the entire task subtree. Since controllers cannot be enabled in populated cgroups, the proxy process is first moved into a dedicated sub-cgroup.

Listing 3.13 Pseudocode of `proxy_cgroup_manager`

```
class proxy_cgroup_manager:
    def __init__(self):
        self.root_cgrp = root_cgroupv2_t()
        self.leaf_cgrp = None

    def run(self):
        self.leaf_cgrp = cgroupv2_t("proxy_leaf")
        if not self.leaf_cgrp.add_me():
            terminate("Failed to move proxy process into proxy_leaf cgroup")
        self.root_cgrp.enable_all_ctrlrs()
```

Class Members:

- `root_cgrp`: Interface to the root of the proxy cgroup namespace (`/sys/fs/cgroup` after the namespace switch).
- `leaf_cgrp`: Dedicated leaf cgroup where the proxy process is relocated before enabling controllers.

Overview of operation:

1. Upon entry, the proxy process initially resides in the root cgroup of its private cgroup namespace.

2. The `run()` method creates a new sub-cgroup (`proxy_leaf`) and moves the proxy process into it.
3. With the root cgroup now empty, controllers can be enabled via `enable_all_cntrlrs()`.
4. This prepares the cgroup hierarchy for task management, where additional cgroups can be created for running user tasks with active controllers (e.g., CPU, memory, PID).

Use of cgroup namespaces:

By combining `CLONE_NEWCGROUP` and `CLONE_INTTO_CGROUP` during proxy startup (see `proxy_connector`, subsection 3.4.5), cgroup management is isolated into a private hierarchy. The proxy controls its subtree independently, isolated from other sandboxes and the host.

3.4.11 Managing sandbox credentials (`proxy_credentials_manager`)

The `proxy_credentials_manager` class provides an interface for managing UID, GID, and related credentials inside the proxy process. It allows switching between the original user and sandbox-specific credentials (the UID/GID pair). The assigned credentials are obtained from the `root_credentials_manager`.

Listing 3.14 Pseudocode of `proxy_credentials_manager`

```
class proxy_credentials_manager:
    def __init__(self, root_manager):
        self.credentials_root = root_manager

    def box_uid(self):
        return self.credentials_root.box_uid()

    def box_gid(self):
        return self.credentials_root.box_gid()

    def box_root(self):
        return self.credentials_root.box_root()

    def box_cgroup(self):
        return self.credentials_root.instance_cgroup()

    def switch_to_user(self):
        setresgid(orig_gid, orig_gid, orig_gid)
        setgroups(0, NULL)
        setresuid(orig_uid, orig_uid, orig_uid)

    def switch_to_box(self):
        setresgid(box_gid, box_gid, box_gid)
        setgroups(0, NULL)
        setresuid(box_uid, box_uid, box_uid)
```

Class Members:

- `credentials_root`: Reference to `root_credentials_manager` providing original and box-assigned credentials.

Overview of operation:

The class offers two main credential switching operations:

- `switch_to_box()`: Switches the real, effective, and saved-set UID and GID to the assigned sandbox UID/GID.
- `switch_to_user()`: Reverts credentials back to those of the launching user.

Obtaining assigned resources:

In addition to credential switching, the class provides access inside the proxy process to sandbox-specific paths (e.g., `box_root()` for filesystem root directory of the sandbox, `box_cgroup()` for the cgroup assigned to this instance).

Switching credentials with `setresuid`, `setresgid` and `setgroups`

Switching credentials inside the proxy is done by a combination of three syscalls: `setresuid()/setresgid()`⁵ and `setgroups()`⁶. These calls are responsible for changing the process real, effective, and saved-set UID/GID, and clearing supplementary groups. The exact order of operations is important to avoid permission errors during the switch.

Listing 3.15 Credential switch implementation

```
void switch_to_box()
{
    auto box_gid = credentials_root->box_gid();
    if(setresgid(box_gid, box_gid, box_gid) < 0)
        { terminate("Couldn't switch to box GID, errno: {}", errno); }

    if(setgroups(0, NULL) < 0)
        { terminate("Setgroups failed, errno: {}", errno); }

    auto box_uid = credentials_root->box_uid();
    if(setresuid(box_uid, box_uid, box_uid) < 0)
        { terminate("Couldn't switch to box UID, errno: {}", errno); }
}
```

3.4.12 Orchestrating tasks (`task_manager`)

The `task_manager` class is responsible for orchestrating the execution of tasks inside the proxy. The actual execution is delegated to the `task_supervisor` class.

⁵<https://man7.org/linux/man-pages/man2/setresuid.2.html>

⁶<https://linux.die.net/man/2/setgroups>

```

class TaskManager:
    def __init__(self, tasks_config, credentials_manager, env_manager):
        self.tasks_config = tasks_config
        self.credentials_manager = credentials_manager
        self.env_manager = env_manager

    def run_all_tasks(self):
        report = TaskReport()

        for task_config in self.tasks_config.get_tasks():
            task = TaskSupervisor(task_config,
                                   self.credentials_manager,
                                   self.env_manager)

            stats = task.run_task()
            report.insert(stats)

        return report

```

In the constructor, the `task_manager` receives:

- The parsed configuration of tasks to execute.
- The credentials manager, responsible for switching to the correct UID/GID for each sandboxed task.
- The environment manager, used to prepare environment variables for each task according to the configuration.

These are then passed to the individual `task_supervisors`.

The main method `run_all_tasks()` iterates over all configured tasks, creates a `task_supervisor` for each one, executes it, and collects execution metadata into a report object.

3.4.13 Supervising task execution (`task_supervisor`)

The `task_supervisor` class implements logic for launching, monitoring, and collecting resource usage data for a single task inside the sandbox. It prepares a separate cgroup for each task, applies requested resource limits, handles standard input/output redirection, switches credentials, executes the configured program, and collects execution statistics including exit codes, resource consumption, and limit violations.

Listing 3.16 Pseudocode of `task_supervisor`

```

class task_supervisor:
    def __init__(self, task_config, credentials, env_manager):
        self.task_config = task_config
        self.credentials = credentials
        self.env_manager = env_manager
        self.task_cgroup = CgroupV2(task_config.name)

    def run_task(self):
        pid = self.launch_task()

```

```

        stats = self.wait_for_task(pid)
        self.task_config.finalize(stats)
        return stats

def launch_task(self):
    fd = self.task_cgroup.open_fd()
    clone_args = self.prepare_clone_args(fd)
    pid = syscall_clone3(clone_args)

    if pid == 0: # child process
        self.optional_chdir()
        self.redirect_descriptors()
        self.set_resource_limits()
        self.credentials.switch_to_box()
        self.call_execve()

        #execve doesn't return on success

        terminate("execve failed")

    return pid

```

Class Members:

- **task_config**: Configuration node for the task
- **credentials**: Reference to the credentials manager to switch UID/GID
- **env_manager**: Reference to the env_manager to generate environment variables according to configuration.
- **task_cgroup**: Interface for the cgroup dedicated for this task.

Overview of operation:

1. **launch_task()** is called to create a child task process using **clone3()**^[14] with **CLONE_INTO_CGROUP**, placing the task into its dedicated cgroup.
2. The child task process applies directory changes, descriptor redirections, resource limits, credentials switching, and finally executes the configured program using **execve()**⁷.
3. The parent process monitors the task's wall-time limit in **wait_for_task()**. If the limit is exceeded, the task is forcibly killed using **SIGKILL**.
4. After termination, **generate_task_stats()** collects full statistics from both the cgroup (memory usage, CPU time) and **getrusage()**^[5].
5. Detected violations (memory limit, CPU limit, wall time limit) are reflected in the returned **exit_status** value.

⁷<https://man7.org/linux/man-pages/man2/execve.2.html>

Resource Limits Enforcement

Task resource limits are applied in two ways:

- Per-process `rlimits`: Set using `setrlimit()` [5], e.g., address space, stack size, open files, file size, core dumps.
- Cgroup limits: Applied via the `cgroupv2_t` interface (described in subsection 3.4.4), limiting total memory usage and number of child processes.

Listing 3.17 Setting Address Space Limit

```
void set_as_size(size_t bytes)
{
    rlimit as{bytes,bytes};
    if(setrlimit(RLIMIT_AS, &as) == -1)
        { terminate("setrlimit() failed"); }
}
```

Redirecting I/O Streams

Before executing the task binary, the proxy allows redirection of standard input, output, and error streams from/to specific files, as specified in the task configuration.

Listing 3.18 `redirect_descriptors()` Implementation

```
void redirect_descriptors()
{
    auto& stdin_f = task_config->stdin_file();
    if(stdin_f)
    {
        if(!std::freopen(stdin_f.value().c_str(), "r", stdin))
            { terminate("Couldn't redirect stdin"); }
    }

    auto& stderr_f = task_config->stderr_file();
    if(stderr_f)
    {
        if(!std::freopen(stderr_f.value().c_str(), "w", stderr))
            { terminate("Couldn't redirect stderr"); }
    }

    auto& stdout_f = task_config->stdout_file();
    if(stdout_f)
    {
        if(!std::freopen(stdout_f.value().c_str(), "w", stdout))
            { terminate("Couldn't redirect stdout"); }
    }

    if(task_config->stderr_to_stdout())
    {
        if(dup2(1, 2) < 0)
            { terminate("dup2() failed while redirecting stderr to stdout."); }
    }
}
```

The `freopen()`⁸ function replaces the file descriptor behind `stdin`, `stdout`,

⁸<https://en.cppreference.com/w/cpp/io/c/freopen>

or `stderr` with the file specified by the configuration. If the configuration does not specify redirection for a stream, the default descriptors inherited from the proxy remain in effect.

The configuration can request combining `stderr` and `stdout` into a single stream. This is implemented via `dup2(1, 2)`⁹, which duplicates file descriptor 1 (`stdout`) into file descriptor 2 (`stderr`), ensuring that both streams are written to the same file descriptor.

Executing the Task Binary

Once the task process has been fully prepared - including environment variable setup, credential switching, resource limit enforcement, and file descriptor redirection - the proxy launches the actual task binary using the `execve()`¹⁰ system call. `execve()` replaces the current process memory with the specified executable, preserving only the process ID and the file descriptor state.

Listing 3.19 `call_execve()` Implementation

```
int call_execve()
{
    auto& exec = task_config->exec_path();
    auto cargs = convert_to_argv(task_config->exec_args());
    char** environ = env_manager->get_envp();

    return execve(exec.c_str(), cargs.data(), environ);
}
```

The implementation constructs the `argv[]` and `envp[]` arrays dynamically based on the task configuration. The `convert_to_argv()` helper function transforms the argument list from configuration into the required C-style array. Similarly, `env_manager` generates the environment variable array.

The `execve()` call only returns in case of failure. Otherwise, the target binary starts immediately. Any error encountered at this stage results in a fatal error, preventing the execution of an incorrectly configured task.

`wait_for_task()` - Task Monitoring Loop

The `wait_for_task()` function implements the monitoring loop that supervises task termination and enforces wall-time limits externally. After launching a task with `clone3()`, the proxy process remains responsible for supervising task execution. Unlike other resource limits, which are enforced automatically by `cgroups` and `rlimits`, wall time requires explicit supervision from the parent process.

The function continuously polls the child process using `waitpid()` with the `WNOHANG` flag to avoid blocking. Each iteration verifies whether the task has terminated, whether it exceeded wall-time, or whether it is still running.

Listing 3.20 `wait_for_task()` Implementation

```
config::task_stats wait_for_task(pid_t task_pid)
```

⁹<https://man7.org/linux/man-pages/man2/dup.2.html>

¹⁰<https://man7.org/linux/man-pages/man2/execve.2.html>

```

{
    int stat{};
    pid_t p;
    auto stime = std::chrono::system_clock::now();
    auto wall_limit = std::chrono::seconds(task_config->rlimits().wall_time())
        ;

    while(true)
    {
        p = waitpid(task_pid, &stat, WNOHANG);
        auto wtime = std::chrono::system_clock::now() - stime;

        if (p < 0)
            { terminate("waitpid() failed."); }
        else if (p == 0)
        {
            if(wtime < wall_limit)
                { std::this_thread::sleep_for(waiting_time()); }
            else
            {
                kill(task_pid, SIGKILL);
                p = waitpid(task_pid, &stat, 0);
                return generate_task_stats(task_cgrp_, stat, wtime,
                    config::exit_status::WALL_TIME_EXCEEDED);
            }
        }
        else
        {
            return generate_task_stats(task_cgrp_, stat, wtime);
        }
    }
}

```

The main logic can be divided into three branches:

1. Error during `waitpid()` (`p == 0`): Any system error from `waitpid()` results in immediate termination.
2. Task still running (`p == 0`): The function compares elapsed wall time with the configured wall-time limit. If the limit has not yet been reached, the function sleeps for a fixed time to avoid busy-waiting. Once wall-time is exceeded, the task receives `SIGKILL` and the proxy calls blocking `waitpid()` to collect its final termination status.
3. Task exited (`p > 0`): In this case, the child terminated normally or via a signal. Exit code, signal, `errno`, wall-time, cgroup resource consumption, and `rusage` statistics are collected and returned as part of the generated `task_stats`.

Collecting Task Execution Statistics

After the task terminates, the proxy collects detailed information about its execution, including resource usage statistics and reason for termination. The information is combined from multiple sources: cgroup statistics, process exit status, and kernel-provided `rusage` structures.

Listing 3.21 generate_task_stats() Implementation

```

config::task_stats generate_task_stats(
    const cgroup::cgroupv2_t& task_cgroup,
    int stat,
    size_t wall_time_ms,
    config::exit_status exit = config::exit_status::OK)
{
    auto r_usage = get_children_rusage();
    auto memory = task_cgroup.memory_usage_bytes();
    auto cputime = task_cgroup.cpu_usage_usec();

    if(task_config->rlimits().memory() && memory > task_config->rlimits().
        memory())
        exit = config::exit_status::MEMORY_LIMIT_EXCEEDED;
    if(task_config->rlimits().cpu_time() && (float)cputime / 1000000 >
        task_config->rlimits().cpu_time().value())
        exit = config::exit_status::CPU_TIME_EXCEEDED;

    return config::task_stats{
        .exited_normally = WIFEXITED(stat),
        .signalled = WIFSIGNALED(stat),
        .exit_code = WEXITSTATUS(stat),
        .err_no = errno,
        .signal = WTERMSIG(stat),
        .exit = exit,

        .cg_total_mem_bytes = memory,
        .cg_total_time_usec = cputime,
        .wall_time_ms = wall_time_ms,

        .rusage_total_mem_bytes = r_usage.ru_maxrss*1000,
        .rusage_total_time_usec = rusage_total_time_usec(r_usage),
    };
}

```

The cgroup interface provides total memory usage and CPU time accounting for the entire task subtree, including subprocesses and threads. This allows detection of memory or CPU time limit violations even if multiple processes contributed to the total resource consumption. Additionally, wall clock time is measured externally by the proxy and compared against the configured wall time limit inside `wait_for_task()`.

The kernel's `rusage` structure extends these measurements with further statistics, such as peak resident set size and CPU user/system times.

4 Testing the ReCoDex Isolator

4.1 Testing Basic Isolation and Resource Limits

Before evaluating real ReCodEx assignments, the container tool was tested in a standalone test suite. The goal was to verify its core functionality: isolation properties and enforcement of resource limits. These tests focus on the expected behavior of Linux primitives (namespaces, cgroups, rlimits) and the filesystem view setup.

The tests are executed using a Python script. The script launches tasks using the Isolator binary, collects output metadata and prints a summary of passing and failing tests. The test suite is located at `tests/test_suite` in the project repository (see Attachment A.1).

Isolation tests

The `isolation_tests/` directory includes testing programs that verify fundamental isolation properties, for example:

- `dir_access_test.cpp` ensures that only the expected files and directories are visible inside the sandbox.
- `env_vars_test.cpp` checks that only configured environment variables are visible.
- `network_isolation_test.cpp` validates that network access is restricted according to the configuration (either full isolation or host namespace).

Resource limits tests

The `limits_tests/` directory contains programs specifically designed to exceed configured limits in a controlled way:

- `cpu_time_test.cpp` loops until it exceeds CPU time.
- `memory_many_allocations.cpp` tries to allocate excessive memory.
- `process_limit_test.cpp` forks until it hits a process count limit.
- `file_size_test.cpp`, `stack_size_test.cpp`, `fd_limit_test.cpp` test other rlimit-based constraints.

Each test is executed twice: once without any limits to ensure it runs successfully, and once with the corresponding limits configured. In the limited run, the expected outcome is for the task to fail due to a specific `exit_status`. This approach verifies both the proper functioning of the programs inside the tool under unrestricted conditions, and the enforcement of limits.

4.2 Testing on ReCoDex assignments

Unlike conventional software, a containerization utility has to handle a variety of runtime environments, compilers, interpreters, and language-specific dependencies. Furthermore, individual assignments may require different libraries, environment variables, or filesystem layouts. This means that it is not feasible to cover the functionality of the tool with manufactured test cases. Since we already have a large ready-to-go set of assignments from ReCoDex, using them as test cases is an obvious choice.

4.2.1 ReCodEx Worker

The means of testing was a script that simulates the ReCoDex Worker component. The ReCodEx worker is a daemon responsible for executing evaluation jobs of solutions submitted to the ReCodEx backend. For each assignment there is a job configuration file which defines a sequence of tasks that must be performed to compile, run, and evaluate a student submission. These tasks form a directed acyclic graph (DAG), with explicit dependencies between them. The worker resolves this DAG and executes the tasks in dependency order.

Each task in the job specification defines a unique `task-id`, optional `dependencies`, and a `cmd` structure that encodes the path to the executable binary and its arguments. Some tasks are marked with "fatal-failure: true", meaning that if they fail, all subsequent tasks are skipped.

The Worker downloads necessary supplementary files, moves source files using internal Worker commands (e.g., `fetch`), and calls compilers or interpreters. For instance, a C# submission may trigger a chain of `mkdir`, `fetch`, `cp`, and `dotnet` invocations to prepare and compile the source. Then the solution is executed on predefined inputs and its outputs are compared against expected outputs using a `Judge` task. Compilation, Execution and Judge tasks are executed in containerized environments.

4.2.2 Mocking the ReCodEx Worker

To emulate the behavior of ReCodEx in a local environment, a Python-based mocking script was developed. The mocking script simulates the ReCodEx Worker in a simplified way. It interprets and executes a ReCodEx job configuration file, managing file fetching, sandboxed task execution and evaluating outputs using Judges.

The testing included assignments from three different environments: C++, Python and C# - to test the varying requirements of compiled, interpreted and JIT-compiled languages.

4.3 Pitfalls of containerized environments

Running real programs using the Isolator tool exposes a range of practical challenges, many of which are caused by the highly configurable nature of the tool.

Most aspects of the sandbox environment, such as visible directories, environment variables, and namespace setup, are determined by user-provided configuration, and correct behavior depends heavily on accurate setup by the user.

Typical problems include missing or misconfigured directory mounts, which can prevent programs from finding required binaries, libraries, or configuration files. Overriding or omitting critical environment variables (like `PATH`) may cause compilers, interpreters, or runtime environments to fail. Namespace isolation can also introduce unexpected behavior: for example, improper mount namespace setup may block access to necessary filesystems, while network or IPC namespaces may restrict communication needed by some tools.

The following sections mention some of the pitfalls revealed when running tests on the ReCoDex assignments. These are not necessarily specific to ReCoDex or our tool.

4.3.1 Compiling C/C++ using gcc/g++

Several unexpected issues were encountered when compiling C++ code. GCC-C/G++ are frontends that coordinate the compilation process and invoke multiple internal tools (such as `cc1`, `cc1plus`, `as`, `ld`). Their behavior is influenced by a combination of environment variables and internal configuration files:

- **Spec files:** Define how GCC invokes internal stages of compilation and linking. They are located in GCC's internal directories (e.g., `/lib/gcc/x86_64-redhat-linux/14/libitm.spec`) and can be viewed using `gcc -dumpspecs`.
- **Environment variables:**
 - `GCC_EXEC_PREFIX`: Overrides the root path for internal executables like `cc1`.
 - `COMPILER_PATH`: Affects where assembler and linker programs are found.
 - `LIBRARY_PATH`: Adds directories to the static linker's search path.
 - `C_INCLUDE_PATH`, `CPLUS_INCLUDE_PATH`: Add include paths for headers.
 - `LD_LIBRARY_PATH`: Affects runtime dynamic linking, not GCC directly.
 - `PATH`: Influences which executables are found when GCC invokes internal tools.

Most of the configuration is handled by GCC itself. GCC prioritizes paths passed via `-I` or `-L`, then environment variables, then built-in defaults. When using a package manager installation of GCC and inheriting the environment from the host system, there are usually no issues. Our tests were also conducted in environments with a non-standard installation of GCC or with some environment variables overwritten, which is when problems can appear.

Incorrect PATH

When PATH is set incorrectly, GCC can fail to find its internal tools, leading to error output like

```
collect2: fatal error: cannot find 'ld'  
compilation terminated.
```

This error seems to only appear when PATH is overridden in the configuration. When inheriting PATH from the host, or not defining PATH at all, the system installation of GCC found its tools without issues.

Incorrect LD_LIBRARY_PATH

There is a variety of error messages we can get when the LD_LIBRARY_PATH variable is incorrectly configured. Usually, it will lead to a missing shared library, like

```
/usr/bin/ld: error while loading shared libraries: libstdc++.so.6: cannot open  
shared object file: No such file or directory
```

However, that is not the only type of issue that can arise. The most obscure error encountered was the following:

```
/opt/rh/gcc-toolset-14/root/usr/libexec/gcc/x86_64-redhat-linux/14/ld: /lib  
/..lib64/libctf.so.0: version LIBCTF_1.1 not found  
(required by /opt/rh/gcc-toolset-14/root/usr/libexec/gcc/x86_64-redhat-linux  
/14/ld)  
collect2: error: ld returned 1 exit status
```

This error happened while calling a custom installation of G++ 14 with a standard installation of G++ 11 on the system. An overwritten LD_LIBRARY_PATH caused G++ to find the standard directories of GCC 11 and its libctf.so.0, which was incompatible with the G++ 14 linker.

The approach taken in the mocking script is to inherit PATH from the host system and generate LD_LIBRARY_PATH based on parsing specific outputs of the gcc/g++ -print-search-dirs command. This solution led to successful compilation of C/C++ code on multiple machines, without needing additional setup.

4.3.2 Offline compilation of C# .NET

By default, the .NET SDK requires connection to the internet, to attempt to download missing dependencies, NuGet packages, or workload components during the build process. This behavior is problematic if we require the compilation itself to be offline.

To allow offline compilation, ReCoDex uses a special .NET setup with a custom installation of the .NET SDK and a dotnet command line explicitly pointing to Roslyn/csc.dll. This setup has been extensively used and works reliably - for the purposes of local testing with our Worker mocking script, it was only necessary to install .NET into the ReCoDex specific /opt/dotnet.

An example of the command line used for offline compilation is as follows (note that in practice, many more files are explicitly specified):

```
bin: /usr/bin/dotnet
args:
  - /opt/dotnet/sdk/latest/Roslyn/bincore/csc.dll
  - -out:Program.exe
  - -target:exe
  - -main:CodEx.Wrapper
  - -r:/opt/dotnet/shared/Microsoft.NETCore.App/latest/Microsoft.CSharp.dll
  - -r:/opt/dotnet/shared/Microsoft.NETCore.App/latest/Microsoft.VisualBasic.
    Core.dll
  - -r:/opt/dotnet/shared/Microsoft.NETCore.App/latest/Microsoft.VisualBasic.
    dll
  - -r:/opt/dotnet/shared/Microsoft.NETCore.App/latest/Microsoft.Win32.
    Primitives.dll
  # ... many more lines ...
```

5 Future work

The primary focus of this work has been the research of available options and delivering a working proof-of-concept. In this chapter, we propose areas for future work to make the tool more robust and production-ready, and also an example of a useful new feature - networking between sandboxes.

Improving Security

The current handling of filesystem permissions and ownership is undeniably not secure. Writable directories for the sandbox are currently created with all permission bits enabled (mode 0777) to allow subsequent instances of the Isolator to access them. This is necessary for ReCoDex jobs, where different phases of a job, such as compilation, execution, and evaluation, run as separate Isolator invocations but need access to the same files. This means potentially any process on the machine can write to or read from these shared directories.

The permission model of the Isolator has to be reviewed and should implement controlled transfer or sharing of ownership. This might mean group-based access permissions or using ACLs¹ for related Isolator instances.

Integration into ReCodEx

ReCoDex is currently configured to work with the CLI interface and output formats of Isolate. For our new tool, we have already performed testing with real ReCoDex job configurations, however, the setup was greatly simplified and the ReCoDex Worker mocking script was already adapted accordingly.

The Worker has to correctly invoke the new tool and interpret its results, adjustments to code on both sides will likely be necessary. Integration will then also require extensive testing with real ReCoDex workloads.

Synchronized Allocation of Instance IDs

In the current implementation, Isolator instances are identified by user-defined or randomly generated IDs, and the only way to reliably prevent collisions is careful configuration from the user's side.

To solve this, a centralized instance ID allocator could be implemented. It would provide atomic allocation and release of instance IDs using Linux synchronization primitives. Before further setup, each isolator invocation would first communicate with this allocator to reserve a unique ID.

Installation and Integration with systemd

Currently, the tool consists of a binary with a few supporting scripts for initialization and cleanup. This is sufficient for local testing and experimentation, but production use would require proper integration into the system.

¹<https://man7.org/linux/man-pages/man5/acl.5.html>

We propose a more structured installation process, including:

- Installation of the binary and configuration files to standard locations (e.g., `/usr/bin`, `/etc/rcdx_cntnr`).
- Integration of helper scripts into the same command as the binary.
- Systemd units for initialization of system resources and the allocator service.

Running without Root Privileges

In the current design, root privileges are assumed throughout the whole runtime of the tool to configure cgroups, mount directories, set file permissions, etc. Knowing that *rootless* containerization tools are possible, such as Podman, this is undesirable.

We already proposed an allocator service for Instance IDs. This could be extended to a service running with root privileges that would perform all necessary privileged steps and delegate resources to an unprivileged instance of the Isolator. This service would be responsible for preparing resources such as:

- Filesystem subtrees for the sandbox
- Cgroups with appropriate ownership, manageable by an unprivileged process
- User namespaces with UID/GID mappings

Individual instances could then be run without root and only request privileged setup by communicating with this allocator service. This would greatly improve the security and versatility of our tool - regular users could create their own sandboxes, without the security risks of running privileged processes.

Controlled Networking Between Sandboxes

Currently, the Isolator supports two modes of network configuration: complete isolation with a separate network namespace (providing only a loopback interface), or full access to the host network namespace. The first option prevents any network communication, the second allows full access to the network of the host.

To allow assignments that require communicating over the network, such as web applications, we might want to allow a middle ground - a controlled restricted network between sandboxes. This would include connecting separate network namespaces with `veth`² pairs, besides other networking configuration.

²<https://man7.org/linux/man-pages/man4/veth.4.html>

Conclusion

This thesis presented the design and implementation of a new containerization tool tailored for the needs of the ReCoDex automated evaluation system. The work was motivated by the limitations of existing solutions and the need for a lightweight, flexible, and extensible isolation mechanism that leverages modern Linux kernel features such as namespaces and cgroupv2.

A significant portion of the work was devoted to extensive research and experimentation with Linux containerization primitives. The lack of comprehensive documentation for many kernel features required a trial-and-error approach, involving detailed study of the interaction of different kernel subsystems and system calls. This process not only influenced the development of the tool, but also provided valuable insights about the pitfalls of building containerized environments.

The resulting tool serves as a working proof-of-concept, demonstrating a modular and well-documented isolation utility specifically engineered for ReCoDex. It has been successfully tested on real assignment workloads, showing its ability to handle diverse runtime environments and enforce resource limits and isolation policies. However, some security aspects, such as the handling of filesystem permissions and ownership, remain to be addressed. The current implementation prioritizes functionality and extensibility, with the understanding that a thorough review of security aspects is necessary before production deployment.

Integration with ReCoDex is underway, with initial testing performed using a mocking script simulating the ReCoDex Worker component. Further work will focus on seamless integration and testing with real workloads to ensure reliability in the production environment.

In summary, this thesis lays a solid foundation for a modern, lightweight, and maintainable isolation mechanism for automated evaluation in ReCoDex. The tool's modular design and extensive documentation make it well suited for future extensions and adaptation to evolving requirements, while the research and experimentation contributed practical insights regarding Linux containerization.

Bibliography

1. MARTIN MAREŠ, BERNARD BLACKHAM. *Isolate - Sandbox for securely executing untrusted programs* [online]. 2025. [visited on 2025-06-24]. Available from: <https://github.com/ioi/isolate>. GitHub repository.
2. MARTIN KRULIŠ. *ReCodEx: Automated Evaluation System* [online]. 2025. [visited on 2025-06-24]. Available from: <https://github.com/recodex>. GitHub repository.
3. MARTIN KRULIŠ. *ReCodEx Worker: Backend part of ReCodEx used for evaluation* [online]. 2024. [visited on 2025-06-24]. Available from: <https://github.com/ReCodEx/worker>. GitHub repository.
4. MARTIN MAREŠ. *Security of grading systems* [online]. 2021. [visited on 2025-06-25]. Available from: <https://mj.ucw.cz/papers/secgrad.pdf>.
5. FREE SOFTWARE FOUNDATION, INC. *The GNU C Library (glibc) manual - Resource Usage and Limitation* [online]. 2023. [visited on 2025-06-25]. Available from: https://www.gnu.org/software/libc/manual/html_node/Resource-Usage-And-Limitation.html.
6. THE LINUX KERNEL DOCUMENTATION PROJECT. *Control Groups Version 2* [online]. The Linux Kernel, 2015. [visited on 2025-06-18]. Available from: <https://docs.kernel.org/admin-guide/cgroup-v2.html>.
7. MICHAEL KERRISK. *namespaces(7) — Overview of Linux namespaces* [online]. Linux man-pages project, 2024. [visited on 2025-06-18]. Available from: <https://man7.org/linux/man-pages/man7/namespaces.7.html>.
8. MICHAEL KERRISK. *mount_namespaces(7) — Overview of Linux mount namespaces* [online]. Linux man-pages project, 2024. [visited on 2025-06-18]. Available from: https://man7.org/linux/man-pages/man7/mount_namespaces.7.html.
9. MICHAEL KERRISK. *cgroup_namespaces(7) — Overview of Linux cgroup namespaces* [online]. Linux man-pages project, 2024. [visited on 2025-06-18]. Available from: https://man7.org/linux/man-pages/man7/cgroup_namespaces.7.html.
10. MICHAEL KERRISK. *pid_namespaces(7) — Overview of Linux PID namespaces* [online]. Linux man-pages project, 2024. [visited on 2025-06-18]. Available from: https://man7.org/linux/man-pages/man7/pid%5C_namespaces.7.html.
11. MICHAEL KERRISK. *network_namespaces(7) — Overview of Linux network namespaces* [online]. Linux man-pages project, 2024. [visited on 2025-06-25]. Available from: https://man7.org/linux/man-pages/man7/network_namespaces.7.html.
12. MICHAEL KERRISK. *user_namespaces(7) — Overview of Linux user namespaces* [online]. Linux man-pages project, 2024. [visited on 2025-06-25]. Available from: https://man7.org/linux/man-pages/man7/uer_namespaces.7.html.

13. MICHAEL KERRISK. *mount(2)* - *Linux manual page* [online]. Linux man-pages project, 2024. [visited on 2025-06-25]. Available from: <https://man7.org/linux/man-pages/man2/mount.2.html>.
14. MAN.ARCHLINUX.ORG. *clone3(2)* — *Linux manual page* [online]. 2025. [visited on 2025-06-18]. Available from: <https://man.archlinux.org/man/clone3.2.en>.
15. MICHAEL KERRISK. *pivot_root(2)* - *Linux manual page* [online]. Linux man-pages project, 2024. [visited on 2025-06-18]. Available from: https://man7.org/linux/man-pages/man2/pivot_root.2.html.

A Attachments

A.1 ReCoDex Isolator project repository

The submitted thesis includes an attachment with a ZIP archive that contains the complete implementation of the developed tool, including helper scripts, configuration files, and test cases.

The contents of the archive correspond to the last state of the project's Git repository (accessible through https://gitlab.mff.cuni.cz/kurzs/rcdx_cntnr) before thesis submission. The structure of the repository is as follows:

- `src/` – Source code of the tool, written in C++ (including the main entry point `main.cpp` and supporting modules such as `cgrps.hpp`, `config.hpp`, etc.).
- `build/` – Build outputs generated by CMake, including the compiled binary (`src/isolator`).
- `scripts/` – Initialization and cleanup scripts (`isolator_init.sh`, `isolator_cleanup.sh`, `isolator_build.sh`).
- `tests/` – Test suite, including:
 - `test_suite/` – Tests for the tool's basic functionality.
 - `recodex/` – Test data and scripts specific to ReCoDex.
- `docs/` – Documentation files, including Doxygen configuration (`Doxyfile`).
- `README.md` – Instructions for building and running the tool.
- `CMakeLists.txt` – CMake build configuration.

A.1.1 Running automated testing scripts

The repository contains two automated sets of tests that can be launched locally without additional configuration.

Basic Test Suite

The `tests/test_suite/` directory contains a test suite that verifies basic isolation behavior and resource limit enforcement. The suite is divided into two main groups:

- `isolation_tests.yml` – Tests for environment isolation (directory access, environment variables, networking).
- `resource_limits_test.yml` – Tests for resource limits (memory, CPU time, number of processes, etc.).

Each test group is defined via a YAML configuration file. Tests are executed using the `run_tests.py` script, which performs system initialization, builds test binaries, and launches all test cases using the `isolator` binary.

```
cd tests/test_suite
sudo ./run_tests.py
```

The script performs several important steps:

- Builds necessary binaries using the `isolator_build.sh` script
- Cleans up previous test artifacts
- Initializes the isolation environment
- Runs both isolation and resource limits tests
- Reports detailed test results, showing which tests passed or failed

For resource limit tests, each test is first executed without resource limits to verify that it would succeed under normal circumstances, and then with limits to verify proper enforcement of resource restrictions.

ReCoDex Tests

The repository also includes special tests for ReCoDex integration in the `tests/recodex/` directory. These tests verify the compatibility of the Isolator with the Recodex system for various programming languages (C++, Python, C#, Java).

To run the Recodex tests:

```
cd tests/recodex
# Initialize the environment
sudo ./recodex_init.py

# Run the tests
./recodex_mock.py [language_group]
```

Where `[language_groups]` can be a subset of "C#", "Python", "C++", or "AdvC++" to test assignments from specific courses.

The mock script will:

- Set up the appropriate environment for each language
- Run reference solutions from ReCoDex (which are supposed to succeed) through the Isolator
- Report any failed solutions

Note: The ReCoDex test suite performs extensive downloads and setup and the C# group of assignments can take tens of minutes to finish.

A.1.2 Running the tool manually

For comprehensive information about the Isolator, including installation, configuration, and usage instructions, please refer to the `README.md` file in the project root directory.

For quick reference, this is how to invoke the Isolator manually:

```
sudo ./build/src/isolator --yaml=/path/to/config.yml
```

Before running the Isolator, run the initialization and build scripts:

```
# Clean up any previous isolator state
sudo ./scripts/isolator_cleanup.sh

# Initialize the isolator environment
sudo ./scripts/isolator_init.sh

# Build the binary
./scripts/isolator_build.sh
```

Configuration File Format

The Isolator uses YAML configuration files to define tasks and their execution environment. Here is a basic example of a configuration file:

```
box-fs:
  dir-rules:
    - "build=/home/user/project/build"
    - "res=/tmp:rw"

tasks:
  - task-id: "example-task"
    chdir: "/res"
    stats-yaml: "/res/example-task.result.yaml"
    cmd:
      bin: "/build/myprogram"
      args: ["arg1", "arg2"]
    limits:
      mem: 128000 # Memory limit in bytes
      cpu-time: 10 # CPU time limit in seconds
      wall-time: 30 # Wall time limit in seconds
```

Key configuration elements:

- **box-fs:** Defines the filesystem rules for the isolation container
 - **dir-rules:** Specifies mappings from the host filesystem to the container
- **tasks:** List of tasks to execute
 - **task-id:** Unique identifier for the task
 - **chdir:** Working directory inside the container (relative to sandbox root)
 - **stats-yaml:** Path to write execution statistics (relative to sandbox root)

- **cmd**: Command to execute
 - * **bin**: Path to the binary (relative to sandbox root)
 - * **args**: Command-line arguments
- **limits**: Resource limitations for the task
 - * Supports various resource constraints (memory, CPU time, etc.)

Directory Rules

The Isolator creates a secure sandbox environment with a strictly controlled filesystem. The **box-fs** section in the configuration defines how the filesystem should be structured within the sandbox.

A simple example:

```
box-fs:
  dir-rules:
    - "bin" # Mount /bin as read-only
    - "tmp=/tmp/mytmp:rw" # Mount with read-write access
    - "build=/home/user/project/bin"
```

These rules create a sandbox where:

- The host's **/bin** directory is available at **/bin** in the sandbox (read-only)
- The host's **/tmp/mytmp** directory is available at **/tmp** with read-write access
- The directory at **/home/usr/project/bin** is mounted at **/build**

Paths in Task Configuration

Important: All paths specified within a task configuration are relative to the sandbox root, not the host filesystem. This is crucial to understand when configuring:

- **chdir** - Working directory for the command (relative to sandbox root)
- **cmd.bin** - Path to the executable (relative to sandbox root)
- **stats-yaml** - Output file for statistics (relative to sandbox root)
- **stdin/stdout/stderr** - I/O file paths (relative to sandbox root)

For example, if we have the following directory rule:

```
box-fs:
  dir-rules:
    - "build=/home/user/myproject/build"
    - "res=/home/user/myproject/results:rw"
```

Then the task configuration would reference these paths as:

```
tasks:
- task-id: "example-task"
  chdir: "/res" # Inside the sandbox at /res
  stats-yaml: "/res/stats.yaml" # Save results to /res/stats.yaml
  cmd:
    bin: "/build/myprogram" # Run /build/myprogram
    args: ["input.txt"]
```

The executable `/build/myprogram` corresponds to `/home/user/myproject/build/myprogram` on the host, and the working directory `/res` corresponds to `/home/user/myproject/results` on the host.